UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN



Listas simplemente enlazadas en Java

Informe del caso de estudio y desarrollo de las operaciones

Algoritmos y Estructura de Datos

Alumno: Quispe Tica Nayeli

Semana 10

Sistema de gestión de alumnos

Modelo · DAO · Vista

Informe basado en el proyecto CSemana10

Código original explicado · Diagramas · Pruebas de escritorio

2026

Contenido

1 Presentación de la clase 4
1.1 Aprendizajes esperados 4
1.2 Ruta sugerida para el laboratorio 4
1.3 Cómo leer los ejemplos 4
2 Caso de estudio 5
2.1 Enunciado 5
2.2 Aplicación al sistema de alumnos 5
2.3 Reglas que ya aplica el programa 5
3 Fundamentos de la lista simplemente enlazada 7
3.1 Qué es una lista simplemente enlazada 7
3.2 Dato, nodo y lista 7
3.3 Referencias en Java 7
3.4 Cómo se reconoce el final y el vacío 8
3.5 Invariantes que deben conservarse 8
3.6 Condiciones y operadores que aparecen en el código 8
3.7 Parámetros, argumentos y retornos 8
4 Organización del proyecto 9
4.1 Archivos del proyecto 9
4.2 Flujo de una operación 9
5 Clases del modelo 10
5.1 Clase Alumno 10
5.2 Lectura del constructor y sus validaciones 11
5.3 Getters, setters y representación del dato 11
5.4 Clase CNodo 12
5.5 Cómo se construye y se conecta un nodo 12
6 Estado inicial y métodos auxiliares 13
6.1 Atributos y constructor de CListaSE 13
6.2 Auxiliar validarAlumnoNuevo 13
6.3 Auxiliar buscarNodo 14
6.4 Datos base de las pruebas 14
6.5 Experimento de identidad de objetos 15
7 Inserción de alumnos 17
Opción 01 Insertar alumno al inicio 17
Opción 02 Insertar alumno al final 18
Opción 03 Insertar alumno en posición X 20
Opción 04 Insertar después de una referencia 22
8 Eliminación de alumnos 24

Opción 05 Eliminar alumno al inicio 24
Opción 06 Eliminar alumno al final 25
Opción 07 Eliminar alumno de posición X 27
Opción 08 Eliminar alumno por referencia 28
9 Búsqueda y recorrido 31
Opción 09 Buscar alumno por código 31
Opción 10 Mostrar todos los alumnos 32
10 Ordenamiento de alumnos 34
10.1 Código compartido de las opciones 11 y 12 34
Opción 11 Ordenar alumnos por código 35
Opción 12 Ordenar alumnos por promedio 35
11 Intercambio de alumnos 37
Opción 13 Intercambiar dos alumnos 37
12 Tamaño, modificación y estado de la lista 39
Opción 14 Mostrar cantidad de alumnos 39
Opción 15 Modificar datos de un alumno 40
Opción 16 Verificar si la lista está vacía 41
13 Funcionamiento de la Vista 43
13.1 Repetición del menú y manejo de avisos 43
13.2 Apertura y ejecución del proyecto 43
13.3 Responsabilidad de cada auxiliar de entrada 43
13.4 Condiciones de la Vista que ayudan al caso 43
13.5 Ejemplos de entradas inválidas 44
14 Laboratorio guiado 45
14.1 Práctica integrada con todas las operaciones 45
14.2 Código didáctico de la práctica integrada 45
14.3 Casos límite que se deben justificar 46
15 Repaso y conclusiones 47
15.1 Comparación de las operaciones 47
15.2 Preguntas de repaso con respuesta 47
15.3 Conclusiones del caso 48
16 Código completo de la Vista 49
16.1 MenuOpciones 49
16.2 Principal 49
17 Materiales de referencia 54

1 Presentación de la clase

Este informe explica cómo funciona la lista simplemente enlazada del sistema de gestión de alumnos. Su propósito es que el estudiante pueda seguir cada referencia, interpretar las condiciones del código y justificar el resultado de una operación antes de ejecutarla.

El desarrollo se basa en las cinco clases Java del archivo CSemana10.zip. La lista se almacena mediante nodos CNodo y referencias siguiente; las operaciones se encuentran en CListaSE. La Vista (Principal y MenuOpciones) solicita los datos y presenta los resultados. Los ejemplos con Mía, Ana, Luis, Sol, Iván y Lucía son datos didácticos para realizar las pruebas de escritorio.

1.1 Aprendizajes esperados

1. Distinguir el objeto Alumno, el nodo CNodo y la lista CListaSE.
2. Representar el estado de inicio, siguiente y cantidad antes y después de cada operación.
3. Explicar los parámetros, retornos, condiciones y recorridos de los métodos del proyecto.
4. Insertar y eliminar conservando la continuidad de la cadena de nodos.
5. Diferenciar cambios de enlaces, cambios de datos y consultas que no modifican la lista.
6. Comprobar resultados mediante pruebas de escritorio y ejecución del código.

1.2 Ruta sugerida para el laboratorio

El enunciado indica un tiempo estimado de 180 minutos. La siguiente distribución es una propuesta de estudio para ese tiempo; no agrega requisitos al caso original.

Actividad	Minutos	Producto esperado
Enunciado y fundamentos	25	Identificar datos, operaciones y referencias.
Modelo y organización del proyecto	25	Reconocer Alumno, CNodo, CListaSE y Vista.
Inserciones	35	Trazar las cuatro formas de insertar.
Eliminaciones	30	Trazar las cuatro formas de retirar nodos.
Búsqueda y demás operaciones	35	Aplicar consultas, ordenamientos y modificación.
Pruebas y repaso	30	Verificar casos normales y casos límite.
Total	180	Desarrollo del caso de estudio.

1.3 Cómo leer los ejemplos

Cada operación presenta su contrato (qué recibe y qué devuelve), el código original, una explicación por línea y una prueba de escritorio. Los números de línea son locales a cada fragmento: ayudan a estudiar el código y no reemplazan las posiciones reales del archivo. En los diagramas, N1, N2, N3… nombran nodos; A01, A02 y A03 identifican objetos Alumno. Esas etiquetas no son direcciones de memoria.

2 Caso de estudio

2.1 Enunciado

Desarrollar un programa en lenguaje Java que utilice Listas Simplemente Enlazadas y que realice las operaciones básicas de:

1. Inserción de elementos: al inicio de la lista, al final de la lista, en una posición x y por referencia.
2. Eliminación de elementos.
3. Búsqueda de elementos.
4. Recorrido de la lista.
5. Ordenamiento de la lista.
6. Intercambio de elementos.
7. Obtener el tamaño de la lista.
8. Modificación de un elemento.

Tiempo estimado para realizar este laboratorio: 180 minutos.

2.2 Aplicación al sistema de alumnos

El proyecto concreta el elemento del enunciado como un Alumno con código, nombre, edad y promedio. Cada CNodo guarda una referencia a ese Alumno y una referencia al siguiente nodo. El sistema permite operar sobre la colección desde un menú de consola.

Los ocho requisitos del enunciado se distribuyen en 16 opciones de trabajo. Las opciones 11 y 12 usan el mismo método ordenar con valores distintos de su parámetro. La opción 16 agrega la consulta de lista vacía y la opción 0 sirve para salir.

Requisito del enunciado	Opciones del menú	Método del proyecto
1 Inserción	1 a 4	insertarInicio, insertarFinal, insertarEnPosicion, insertarPorReferencia
2 Eliminación	5 a 8	eliminarInicio, eliminarFinal, eliminarEnPosicion, eliminarPorReferencia
3 Búsqueda	9	buscarPorCodigo
4 Recorrido	10	mostrarTodos
5 Ordenamiento	11 y 12	ordenar(false) y ordenar(true)
6 Intercambio	13	intercambiar
7 Tamaño	14	obtenerTamanio
8 Modificación	15	modificar
Consulta adicional	16	estaVacia

2.3 Reglas que ya aplica el programa

1. El código identifica al alumno, se normaliza a mayúsculas y no puede repetirse en la lista.
2. El nombre es obligatorio, la edad debe ser positiva y el promedio debe ser finito y estar entre 0 y 20.

3. Las posiciones se cuentan desde 1. Al insertar se admite hasta cantidad + 1; al eliminar, hasta cantidad.
4. La inserción por referencia agrega después del mismo objeto Alumno recibido.
5. La modificación conserva el código. Los ordenamientos y el intercambio cambian los datos guardados en los nodos, no los enlaces.

3 Fundamentos de la lista simplemente enlazada

3.1 Qué es una lista simplemente enlazada

Es una estructura lineal formada por nodos. Cada nodo contiene un dato y un enlace hacia el nodo siguiente. Se llama simplemente enlazada porque cada CNodo de este proyecto tiene un único enlace de avance: siguiente. El último nodo termina en null.

La lista es dinámica: al insertar se crea un CNodo y se conecta a la cadena. El proyecto no reserva un arreglo de alumnos ni usa ArrayList para almacenarlos. Para alcanzar un alumno ubicado más adelante, se avanza por los enlaces desde inicio.



Figura 1. La cadena se recorre desde inicio siguiendo los enlaces. A01 representa la referencia al Alumno de N1.

3.2 Dato, nodo y lista

Componente	Qué contiene	Responsabilidad
Alumno	codigo, nombre, edad y promedio	Representar y validar los datos de una persona.
CNodo	alumno y siguiente	Guardar la referencia al dato y al próximo nodo.
CListaSE	inicio y cantidad	Administrar la cadena y sus operaciones.

CNodo no tiene un campo codigo propio. Cuando un diagrama muestra A01 dentro del nodo, resume el Alumno al que apunta su campo alumno. El código se obtiene con getAlumno().getCodigo().

3.3 Referencias en Java

Una variable de tipo CNodo o Alumno guarda una referencia a un objeto. Asignar una referencia a otra variable no copia el objeto completo: dos variables pueden señalar al mismo nodo o al mismo alumno.

Tres instrucciones con efectos diferentes

```
01  CNodo actual = inicio;
02  actual = actual.getSiguiente();
03  actual.setSiguiente(nuevo);
```

1. CNodo actual = inicio copia la referencia al primer nodo. inicio y actual pueden señalar el mismo CNodo.
2. actual = actual.getSiguiente() cambia el lugar que recorre la variable local actual. No cambia por sí sola ningún enlace.
3. actual.setSiguiente(nuevo) cambia el campo siguiente del nodo al que actual señala. Esa instrucción sí modifica la estructura.

3.4 Cómo se reconoce el final y el vacío

inicio == null indica que no hay primer nodo y, por tanto, la lista está vacía. En cambio, nodo.getSiguiente() == null indica que ese nodo es el último. Una lista con un único nodo tiene inicio distinto de null y el siguiente de ese nodo igual a null.

Estado	inicio	cantidad	Último siguiente
Vacía	null	0	No existe último nodo.
Un alumno	Referencia al único CNodo	1	null
Varios alumnos	Referencia al primero	Número de nodos	null

3.5 Invariantes que deben conservarse

Un invariante es una condición que debe seguir siendo verdadera después de una operación válida. En esta implementación, cantidad coincide con el número de nodos alcanzables desde inicio; el último siguiente es null; los alumnos de la lista no son null y sus códigos son únicos. Los métodos de inserción y eliminación mantienen estas condiciones si la lista estaba válida antes de llamarlos.

3.6 Condiciones y operadores que aparecen en el código

Expresión	Lectura sencilla	Uso en el proyecto
x == null	No existe el objeto referido por x.	Detectar vacío o un resultado no encontrado.
x != null	Sí existe un objeto en esa referencia.	Recorrer mientras haya un nodo.
A && B	Deben cumplirse ambas condiciones.	Comprobar actual != null y luego consultar su alumno.
A || B	Basta que se cumpla una.	Rechazar una posición menor que 1 o mayor que el límite.
ref1 == ref2	Señalan el mismo objeto.	Eliminar o insertar por identidad de Alumno.
equalsIgnoreCase	Coincide el texto sin distinguir mayúsculas.	Buscar por código.
cond ? valorA : valorB	Elegir un resultado según true o false.	Seleccionar el criterio de ordenamiento.

Los operadores && y || evalúan de izquierda a derecha con cortocircuito. En actual != null && actual.getAlumno() != referencia, si actual es null, Java ya no evalúa la segunda parte; así se evita consultar un método sobre una referencia null.

3.7 Parámetros, argumentos y retornos

El parámetro es la variable declarada en la cabecera del método; el argumento es el valor que se entrega al llamarlo. En insertarEnPosicion(Alumno alumno, int posicion), alumno y posicion son parámetros. En lista.insertarEnPosicion(a04, 2), la referencia a04 y el número 2 son los argumentos.

Java pasa los argumentos por valor. Cuando el argumento es un objeto, ese valor es una referencia: el método recibe una copia de la referencia y accede al mismo objeto. Un retorno Alumno entrega una referencia; boolean expresa true o false; String entrega texto y void no entrega ningún valor.

4 Organización del proyecto

4.1 Archivos del proyecto

El proyecto se llama CSemana10 y está organizado en tres paquetes: Modelo, DAO y Vista. Los datos viven en Modelo, la lógica de la lista en DAO y la interacción con el usuario en Vista.

Archivo	Paquete	Qué explica esta clase
Alumno.java	Modelo	Datos, validaciones y formato de salida.
CNodo.java	Modelo	Dato del nodo y enlace siguiente.
CListaSE.java	DAO	Estado de la lista y operaciones del caso.
MenuOpciones.java	Vista	Texto del menú de consola.
Principal.java	Vista	Lectura de datos, selección de operaciones y mensajes.

4.2 Flujo de una operación

1. Principal pide los datos al usuario y los convierte al tipo necesario.
2. La Vista construye o busca un Alumno y llama al método de CListaSE.
3. CListaSE valida, recorre o modifica nodos según la operación.
4. El método devuelve el dato que corresponda, o termina si es void.
5. Principal imprime el resultado o captura una IllegalArgumentException y muestra el aviso.

Por ejemplo, la opción 8 pide un código, ejecuta buscarPorCodigo para obtener el objeto almacenado y luego llama a eliminarPorReferencia. De ese modo, la entrada por código de la consola se convierte en una operación por identidad dentro de la lista.

La lista vive en memoria durante la ejecución. El proyecto no incorpora base de datos ni archivo de persistencia: al iniciar otra ejecución, Principal construye una nueva lista vacía.

5 Clases del modelo

5.1 Clase Alumno

Código original completo de Modelo.Alumno

```
01 package Modelo;
02
03 public class Alumno {
04     private final String codigo;
05     private String nombre;
06     private int edad;
07     private double promedio;
08
09     public Alumno(String codigo, String nombre, int edad, double promedio) {
10         if (codigo == null || codigo.trim().isEmpty()) {
11             throw new IllegalArgumentException("El codigo es obligatorio.");
12         }
13         this.codigo = codigo.trim().toUpperCase(java.util.Locale.ROOT);
14         setNombre(nombre);
15         setEdad(edad);
16         setPromedio(promedio);
17     }
18
19     public String getCodigo() { return codigo; }
20     public String getNombre() { return nombre; }
21     public int getEdad() { return edad; }
22     public double getPromedio() { return promedio; }
23
24     public void setNombre(String nombre) {
25         if (nombre == null || nombre.trim().isEmpty()) {
26             throw new IllegalArgumentException("El nombre es obligatorio.");
27         }
28         this.nombre = nombre.trim();
29     }
30
31     public void setEdad(int edad) {
32         if (edad <= 0) {
33             throw new IllegalArgumentException("La edad debe ser positiva.");
34         }
35         this.edad = edad;
36     }
37
38     public void setPromedio(double promedio) {
39         if (!Double.isFinite(promedio) || promedio < 0 || promedio > 20) {
40             throw new IllegalArgumentException("El promedio debe estar entre 0 y 20.");
```

```
41          }
42          this.promedio = promedio;
43      }
44
45      @Override
46      public String toString() {
47          return String.format(java.util.Locale.US,
48                  "Codigo: %s | Nombre: %s | Edad: %d | Promedio: %.2f",
49                  codigo, nombre, edad, promedio);
50      }
51  }
```

5.2 Lectura del constructor y sus validaciones

Alumno representa el dato de la lista. Sus atributos son privados para que las validaciones de sus métodos controlen los valores aceptados. El código es final: una vez construido el objeto no se reasigna mediante un setter.

El constructor recibe codigo, nombre, edad y promedio. Primero rechaza un código null o vacío después de trim. Luego guarda el código sin espacios exteriores y en mayúsculas, con Locale.ROOT. Por último llama a los setters de los otros tres atributos, de modo que aplica sus reglas desde la creación del objeto.

Parte	Condición	Resultado
Código	codigo == null o codigo.trim().isEmpty()	Excepción «El codigo es obligatorio.». Si es válido, queda normalizado.
Nombre	nombre == null o nombre.trim().isEmpty()	Excepción si falta. Si es válido, guarda nombre.trim().
Edad	edad <= 0	Excepción si no es positiva.
Promedio	No finito, menor que 0 o mayor que 20	Excepción. Los valores 0 y 20 sí se admiten.

Double.isFinite también rechaza NaN e infinitos, de modo que el promedio no se limita a comprobar los dos extremos numéricos. Las excepciones evitan construir o modificar un Alumno con un estado inválido.

5.3 Getters, setters y representación del dato

Los getters devuelven los valores de los atributos. Los setters cambian nombre, edad y promedio después de validarlos; no existe setCodigo. Cuando se imprime un Alumno, toString forma el texto con código, nombre, edad y promedio. Locale.US y %.2f hacen que el promedio se vea, por ejemplo, como 16.00.

Ejemplo didáctico de construcción

```
01  Alumno a = new Alumno(" a01 ", " Ana ", 20, 16.0);
02  System.out.println(a);
```

Salida esperada

```
01  Codigo: A01 | Nombre: Ana | Edad: 20 | Promedio: 16.00
```

5.4 Clase CNodo

CNodo aporta la estructura enlazada. No solicita datos por teclado ni imprime mensajes. El campo alumno referencia el objeto con los datos y siguiente referencia otro nodo de la misma clase.

Código original completo de Modelo.CNodo

```
01  package Modelo;
02
03  public class CNodo {
04      private Alumno alumno;
05      private CNodo siguiente;
06
07      public CNodo(Alumno alumno) {
08          this.alumno = alumno;
09          this.siguiente = null;
10      }
11
12      public Alumno getAlumno() { return alumno; }
13      public void setAlumno(Alumno alumno) { this.alumno = alumno; }
14      public CNodo getSiguiente() { return siguiente; }
15      public void setSiguiente(CNodo siguiente) { this.siguiente = siguiente; }
16  }
```

5.5 Cómo se construye y se conecta un nodo

1. private Alumno alumno declara la referencia al dato. private CNodo siguiente declara el enlace de avance.
2. CNodo(Alumno alumno) recibe la referencia al dato; this.alumno = alumno la guarda en el objeto nuevo.
3. this.siguiente = null deja al nodo inicialmente sin sucesor. Todavía no lo conecta con la lista.
4. getAlumno y getSiguiente permiten consultar los campos.
5. setAlumno reemplaza la referencia al dato dentro del mismo nodo. ordenar e intercambiar usan este cambio.
6. setSiguiente reemplaza el enlace al próximo nodo. Las inserciones y eliminaciones usan este cambio.

Crear un CNodo no basta para insertarlo. El método de CListaSE debe hacerlo accesible desde inicio, cambiando inicio o el siguiente de otro nodo. Por eso creación y enlace son pasos distintos.

6 Estado inicial y métodos auxiliares

6.1 Atributos y constructor de CListaSE

Declaración original y estado inicial

```
01  package DAO;
02
03  import Modelo.Alumno;
04  import Modelo.CNodo;
05
06  public class CListaSE {
07      private CNodo inicio;
08      private int cantidad;
09
10      public CListaSE() {
11          inicio = null;
12          cantidad = 0;
13      }
```

inicio señala el primer CNodo y cantidad almacena cuántos nodos pertenecen a la lista. El constructor deja inicio en null y cantidad en 0; no crea un nodo vacío ni un Alumno de relleno. El archivo importa Alumno y CNodo porque usa esas dos clases del paquete Modelo, y los atributos son privados: solo las operaciones públicas de CListaSE administran sus cambios.

6.2 Auxiliar validarAlumnoNuevo

Código original del auxiliar

```
01  private void validarAlumnoNuevo(Alumno alumno) {
02      if (alumno == null) {
03          throw new IllegalArgumentException("El alumno no puede ser null.");
04      }
05      if (buscarPorCodigo(alumno.getCodigo()) != null) {
06          throw new IllegalArgumentException("Ya existe un alumno con ese codigo.");
07      }
08  }
```

Explicación línea por línea

Línea 01. Declara un auxiliar privado. Lo usan las inserciones de la misma clase y recibe al candidato a ingresar.

Línea 02. Comprueba si la referencia alumno es null antes de intentar consultar sus datos.

Línea 03. Lanza una excepción si no hay objeto. La inserción que lo llamó se interrumpe.

Línea 04. Cierra la primera validación.

Línea 05. Busca si el código ya está guardado en otro nodo. Un resultado distinto de null significa que encontró una coincidencia.

Línea 06. Lanza la excepción de código repetido. No crea ni conecta un nodo nuevo.

Líneas 07–08. Cierran la segunda validación y el auxiliar. Si la ejecución llega aquí, el alumno puede continuar su inserción.

Este auxiliar explica el costo de insertarInicio: aunque conectar al principio requiere pocas asignaciones, comprobar duplicados puede visitar todos los nodos. El método completo tiene costo O(n) en el peor caso.

6.3 Auxiliar buscarNodo

Código original del auxiliar

```
01  private CNodo buscarNodo(String codigo) {
02      if (codigo == null) return null;
03      CNodo actual = inicio;
04      while (actual != null) {
05          if (actual.getAlumno().getCodigo().equalsIgnoreCase(codigo.trim())) {
06              return actual;
07          }
08          actual = actual.getSiguiente();
09      }
10      return null;
11  }
```

Explicación línea por línea

Línea 01. Declara una búsqueda privada cuyo resultado es CNodo, no Alumno.

Línea 02. Con un código null devuelve null inmediatamente.

Línea 03. Comienza en el nodo inicial.

Línea 04. Recorre mientras actual señale un nodo real.

Línea 05. Compara los códigos sin distinguir mayúsculas y sin espacios en los extremos del argumento.

Líneas 06–07. Entregan el nodo coincidente. Así intercambiar puede ejecutar setAlumno sobre ese nodo.

Líneas 08–09. Avanzan por el enlace siguiente si todavía no se encontró el código y cierran el recorrido.

Líneas 10–11. Si no hubo coincidencia, devuelve null y se cierra el auxiliar.

La llave final que sigue a estos auxiliares en el archivo cierra la clase CListaSE. Las operaciones públicas se estudian según el orden del menú, aunque los auxiliares estén al final del archivo original.

6.4 Datos base de las pruebas

Código	Nombre	Edad	Promedio
A00	Mía	18	15.0
A01	Ana	20	16.0

Código	Nombre	Edad	Promedio
A02	Luis	21	18.0
A03	Sol	19	14.0
A04	Iván	22	17.0

La lista base contiene A01, A02 y A03 en ese orden. Los métodos siguientes son apoyo didáctico para preparar las pruebas; no forman parte del proyecto original. Se pueden ubicar en una clase de pruebas que importe DAO.CListaSE y Modelo.Alumno.

Preparación didáctica de listas independientes

```
01  private static CListaSE crearListaBase() {
02      CListaSE lista = new CListaSE();
03      lista.insertarFinal(new Alumno("A01", "Ana", 20, 16.0));
04      lista.insertarFinal(new Alumno("A02", "Luis", 21, 18.0));
05      lista.insertarFinal(new Alumno("A03", "Sol", 19, 14.0));
06      return lista;
07  }
08
09  private static CListaSE crearListaParaOrdenar() {
10      CListaSE lista = new CListaSE();
11      lista.insertarFinal(new Alumno("A03", "Sol", 19, 14.0));
12      lista.insertarFinal(new Alumno("A01", "Ana", 20, 16.0));
13      lista.insertarFinal(new Alumno("A02", "Luis", 21, 18.0));
14      return lista;
15  }
```

6.5 Experimento de identidad de objetos

Dos objetos pueden tener el mismo contenido y, aun así, ser objetos distintos. Esta diferencia es la base de insertarPorReferencia y eliminarPorReferencia, que comparan con == y no con equals.

Fragmento didáctico

```
01  CListaSE lista = crearListaBase();
02  Alumno enLista = lista.buscarPorCodigo("A02");
03  Alumno otroA02 = new Alumno("A02", "Luis", 21, 18.0);
04  System.out.println(enLista == otroA02);
05  System.out.println(enLista.getCodigo().equals(otroA02.getCodigo()));
06  System.out.println(enLista == lista.buscarPorCodigo("A02"));
```

Salida esperada

```
01  false
02  true
03  true
```

La primera comparación es false porque otroA02 es un objeto distinto. La segunda es true porque los códigos coinciden. La tercera es true porque buscarPorCodigo devuelve siempre el mismo

objeto almacenado. Por eso, con otroA02 como referencia, insertarPorReferencia lanzaría «La referencia no pertenece a la lista.».

7 Inserción de alumnos

Insertar incorpora un CNodo nuevo y aumenta cantidad. El punto central es conservar la continuación de la cadena antes de sustituir un enlace: primero el nuevo nodo apunta a lo que seguirá, y recién después el nodo anterior apunta al nuevo. Las cuatro variantes comparten la validación del alumno y del código único.

Opción 01 Insertar alumno al inicio

Crea un nodo y lo coloca antes del primer alumno de la lista.

Parámetros. alumno: A00 · Mía · 18 años · 15.0.

Retorno. void: no devuelve un valor.

Código original de insertarInicio

```
01  public void insertarInicio(Alumno alumno) {
02      validarAlumnoNuevo(alumno);
03      CNodo nuevo = new CNodo(alumno);
04      nuevo.setSiguiente(inicio);
05      inicio = nuevo;
06      cantidad++;
07  }
```

Explicación línea por línea

Línea 01. Declara un método accesible desde la Vista. Recibe una referencia de tipo Alumno llamada alumno. void indica que no entrega un valor de retorno.

Línea 02. Llama al auxiliar que rechaza un alumno null y un código ya registrado. Si falla una validación, la ejecución se detiene aquí.

Línea 03. Crea el nodo que guardará la referencia al alumno. nuevo es una variable local que señala ese CNodo; su enlace siguiente empieza en null.

Línea 04. Guarda en nuevo.siguiente la referencia al primer nodo actual. Así se conserva el acceso a todos los alumnos que ya estaban en la lista.

Línea 05. Hace que el atributo inicio señale al nodo nuevo. Desde este momento, el alumno recibido ocupa la primera posición.

Línea 06. Suma uno al contador, porque se incorporó exactamente un nodo. No cuenta los nodos de nuevo.

Línea 07. Cierra el método. Al ser void, puede terminar sin escribir return.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno a00 = new Alumno("A00", "Mía", 18, 15.0);
03  lista.insertarInicio(a00);
```

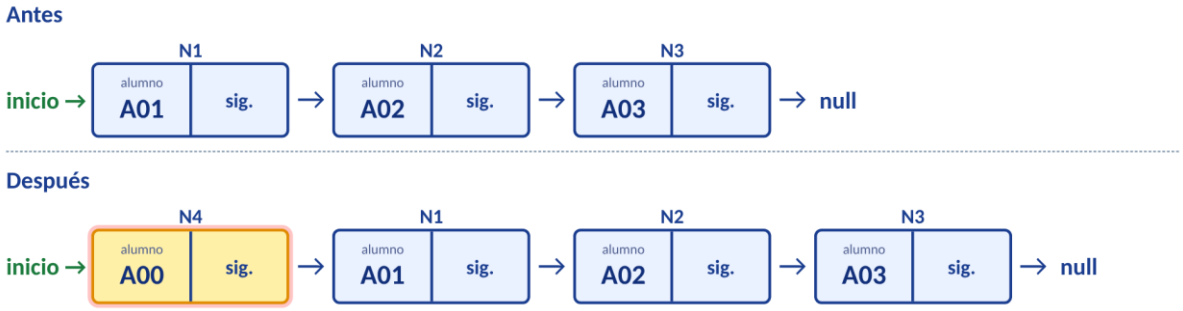


Figura 2. Insertar alumno al inicio. N identifica al nodo y A al Alumno que referencia.

Paso	Qué se hace	Qué se observa
1	Validar y crear	validarAlumnoNuevo no halla A00; se crea N4 con el alumno A00.
2	Enlazar	N4.siguiente guarda la referencia al antiguo primer nodo, N1.
3	Mover inicio	El acceso a la lista comienza ahora en N4.
4	Actualizar cantidad	Pasa de 3 a 4. Se conserva el orden previo: A01, A02 y A03.

Resultado comprobable. A00 → A01 → A02 → A03 | cantidad = 4.

En el menú. Alumno insertado al inicio.

El mensaje de la Vista confirma la acción; para observar el contenido de la cadena se elige después la opción 10. La opción 14 permite comprobar el contador.

Casos que deben revisarse

Lista vacía. nuevo.siguiente queda null; inicio señala al nuevo y cantidad pasa a 1.

Entrada inválida. Un alumno null o un código repetido lanza IllegalArgumentException y no cambia nada.

Costo de la operación. O(n) en total por validar el código; enlazar es O(1).

Contador. cantidad solo aumenta después de completar la inserción.

Opción 02 Insertar alumno al final

Crea un nodo y lo conecta después del último nodo. Si la lista está vacía, el nuevo nodo se convierte en el primero.

Parámetros. alumno: A04 · Iván · 22 años · 17.0.

Retorno. void: no devuelve un valor.

Código original de insertarFinal

```
01  public void insertarFinal(Alumno alumno) {
02      validarAlumnoNuevo(alumno);
03      CNodo nuevo = new CNodo(alumno);
04      if (estaVacia()) {
05          inicio = nuevo;
06      } else {
07          CNodo actual = inicio;
08          while (actual.getSiguiente() != null) {
```

```
09              actual = actual.getSiguiente();
10          }
11          actual.setSiguiente(nuevo);
12      }
13      cantidad++;
14  }
```

Explicación línea por línea

Línea 01. Declara el método con un parámetro Alumno y sin retorno.

Línea 02. Rechaza un alumno null o con código repetido antes de crear nada.

Línea 03. Crea el nodo nuevo; su siguiente es null, lo que ya lo deja listo para ser el último.

Líneas 04–05. Si la lista está vacía, inicio pasa a señalar al nuevo nodo. No hay otro nodo al cual enlazarlo.

Líneas 06–07. Si ya hay nodos, actual empieza en el primero para recorrer la cadena.

Líneas 08–09. Mientras actual.getSiguiente() no sea null, actual avanza. Al terminar, actual señala al último nodo.

Líneas 10–11. Cierra el recorrido y conecta el último nodo con el nuevo mediante setSiguiente.

Líneas 12–13. Cierra la rama else y suma uno a cantidad, tanto si la lista estaba vacía como si no.

Línea 14. Cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno a04 = new Alumno("A04", "Iván", 22, 17.0);
03  lista.insertarFinal(a04);
```

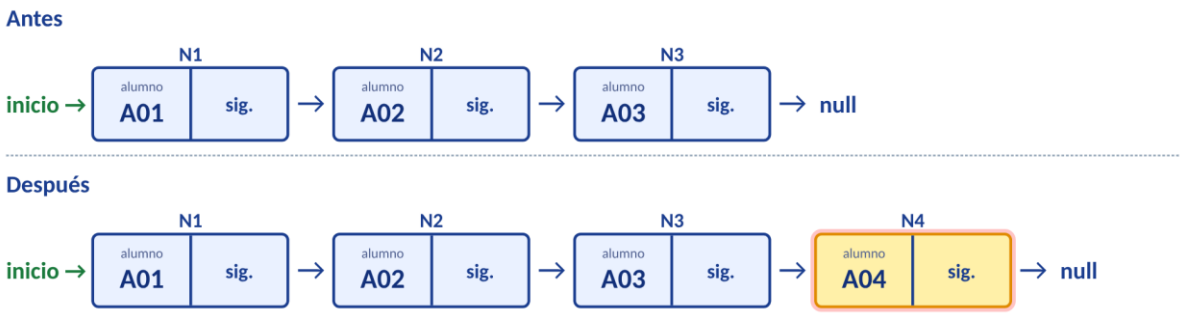


Figura 3. Insertar alumno al final. El último nodo deja de apuntar a null y apunta a N4.

Paso	Qué se hace	Qué se observa
1	Validar y crear	Se crea N4 con A04; N4.siguiente es null.
2	Buscar el último	actual avanza N1 → N2 → N3 y se detiene en N3, porque N3.siguiente es null.
3	Enlazar	N3.siguiente pasa a señalar a N4.

Paso	Qué se hace	Qué se observa
4	Actualizar cantidad	Pasa de 3 a 4.

Resultado comprobable. A01 → A02 → A03 → A04 | cantidad = 4.

En el menú. Alumno insertado al final.

Casos que deben revisarse

Lista vacía. Se ejecuta la rama if: inicio = nuevo. No hay recorrido.

Entrada inválida. Alumno null o código repetido: excepción antes de crear el nodo.

Costo de la operación. O(n) por el recorrido hasta el último nodo; O(1) de espacio auxiliar.

Aplicación de la lista. Como no hay puntero al último nodo, cada inserción al final recorre toda la cadena.

Opción 03 Insertar alumno en posición X

Inserta un alumno de modo que quede en la posición indicada. Las posiciones se cuentan desde 1 y se admite hasta cantidad + 1.

Parámetros. alumno: A04 · Iván · 22 años · 17.0; posicion: 2.

Retorno. void: no devuelve un valor.

Código original de insertarEnPosicion

```
01  public void insertarEnPosicion(Alumno alumno, int posicion) {
02      if (posicion < 1 || posicion > cantidad + 1) {
03          throw new IllegalArgumentException("Posicion valida: 1 a " + (cantidad + 1));
04      }
05      if (posicion == 1) {
06          insertarInicio(alumno);
07          return;
08      }
09      validarAlumnoNuevo(alumno);
10      CNodo anterior = inicio;
11      for (int i = 1; i < posicion - 1; i++) {
12          anterior = anterior.getSiguiente();
13      }
14      CNodo nuevo = new CNodo(alumno);
15      nuevo.setSiguiente(anterior.getSiguiente());
16      anterior.setSiguiente(nuevo);
17      cantidad++;
18  }
```

Explicación línea por línea

Línea 01. Declara el método con el alumno y la posición deseada.

Líneas 02–04. Rechaza posiciones menores que 1 o mayores que cantidad + 1. El mensaje informa el rango válido.

Líneas 05–08. Si la posición es 1, delega en insertarInicio y termina con return. Ese método valida al alumno.

Línea 09. Para las demás posiciones, valida alumno null y código repetido.

Línea 10. anterior comienza en el primer nodo.

Líneas 11–13. Avanza posicion − 2 veces. Con posición 2 no avanza; con posición 3 avanza una vez. Al terminar, anterior es el nodo que quedará antes del nuevo.

Línea 14. Crea el nodo con el alumno recibido.

Línea 15. El nuevo nodo apunta al que hoy sigue a anterior. Este paso conserva el resto de la cadena.

Línea 16. anterior apunta al nuevo nodo. Con esto el nodo queda conectado.

Líneas 17–18. Aumenta cantidad y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno a04 = new Alumno("A04", "Iván", 22, 17.0);
03  lista.insertarEnPosicion(a04, 2);
```

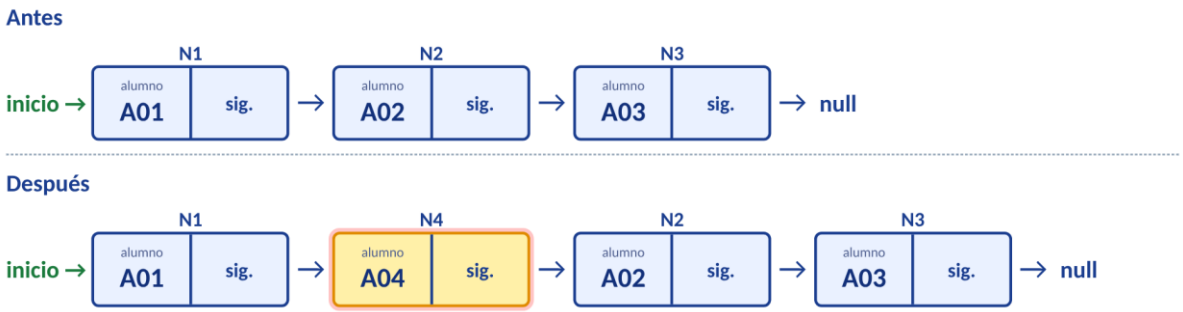


Figura 4. Insertar en la posición 2. N4 queda entre N1 y N2.

Paso	Qué se hace	Qué se observa
1	Validar posición	2 está entre 1 y cantidad + 1 = 4.
2	Ubicar anterior	El for no se ejecuta (1 < 1 es falso); anterior = N1.
3	Enlazar	N4.siguiente = N2 y luego N1.siguiente = N4.
4	Actualizar cantidad	Pasa de 3 a 4.

Resultado comprobable. A01 → A04 → A02 → A03 | cantidad = 4.

En el menú. Alumno insertado en posicion 2

Casos que deben revisarse

Posición 1. Se delega en insertarInicio.

Posición cantidad + 1. El for avanza hasta el último nodo y el nuevo queda al final con siguiente null.

Posición inválida. Con 0 o con 5 (lista de 3) lanza «Posicion valida: 1 a 4».

Orden de enlaces. Primero nuevo.siguiente, después anterior.siguiente. Invertirlos perdería el resto de la cadena.

Costo de la operación. O(n) en el peor caso.

Opción 04 Insertar después de una referencia

Inserta un alumno inmediatamente después de un alumno de referencia que ya pertenece a la lista.

Parámetros. referencia: el objeto Alumno A02; alumno: A04 · Iván · 22 años · 17.0.

Retorno. void: no devuelve un valor.

Código original de insertarPorReferencia

```
01  public void insertarPorReferencia(Alumno referencia, Alumno alumno) {
02      if (referencia == null) {
03          throw new IllegalArgumentException("No existe el alumno de referencia.");
04      }
05      validarAlumnoNuevo(alumno);
06      CNodo actual = inicio;
07      while (actual != null && actual.getAlumno() != referencia) {
08          actual = actual.getSiguiente();
09      }
10      if (actual == null) {
11          throw new IllegalArgumentException("La referencia no pertenece a la lista.");
12      }
13      CNodo nuevo = new CNodo(alumno);
14      nuevo.setSiguiente(actual.getSiguiente());
15      actual.setSiguiente(nuevo);
16      cantidad++;
17  }
```

Explicación línea por línea

Línea 01. Recibe dos referencias: la del alumno existente y la del nuevo.

Líneas 02–04. Si la referencia es null, lanza una excepción. En la Vista, un código inexistente se detecta antes de llegar aquí.

Línea 05. Valida al alumno nuevo (no null y código no repetido).

Línea 06. actual comienza en el primer nodo.

Líneas 07–09. Avanza mientras haya nodo y su alumno no sea el mismo objeto de la referencia. La comparación != es de identidad.

Líneas 10–12. Si actual llegó a null, la referencia no estaba en la lista y se lanza la excepción.

Línea 13. Crea el nodo para el alumno nuevo.

Línea 14. El nuevo apunta al sucesor del nodo de la referencia.

Línea 15. El nodo de la referencia apunta al nuevo.

Líneas 16–17. Aumenta cantidad y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno ref = lista.buscarPorCodigo("A02");
03  Alumno a04 = new Alumno("A04", "Iván", 22, 17.0);
04  lista.insertarPorReferencia(ref, a04);
```

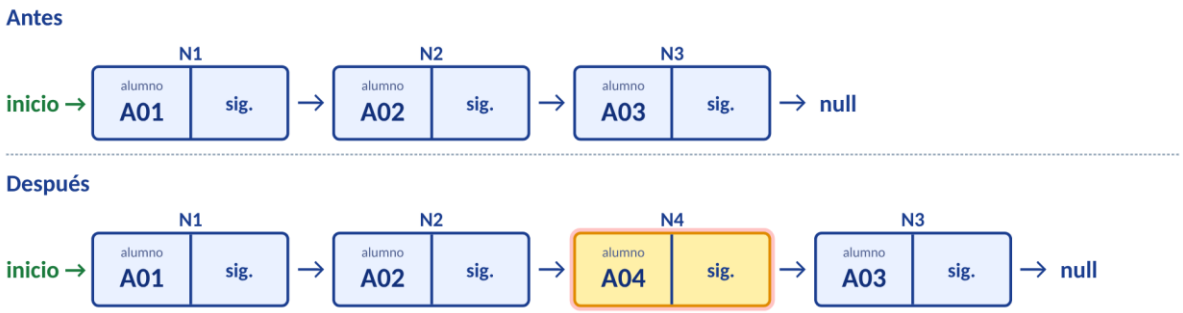


Figura 5. Insertar después de la referencia A02. N4 queda entre N2 y N3.

Paso	Qué se hace	Qué se observa
1	Conseguir la referencia	La Vista obtiene el objeto Alumno que está en N2.
2	Localizar el nodo	actual revisa N1 (no coincide) y N2 (coincide) y se detiene.
3	Enlazar	N4.siguiente = N3 y luego N2.siguiente = N4.
4	Actualizar cantidad	Pasa de 3 a 4.

Resultado comprobable. A01 → A02 → A04 → A03 | cantidad = 4.

En el menú. Alumno insertado despues de A02

Casos que deben revisarse

Identidad del objeto. Otro Alumno con código A02 no es la misma referencia: lanza «La referencia no pertenece a la lista.».

Referencia nula. Lanza «No existe el alumno de referencia.».

Referencia al último. actual.getSiguiente() es null, por lo que el nuevo queda al final.

Costo de la operación. O(n) por la búsqueda; la inserción en sí es O(1).

8 Eliminación de alumnos

Eliminar retira un nodo de la cadena y disminuye cantidad. Los métodos devuelven el Alumno retirado, o null si no había nada que retirar. Para quitar un nodo intermedio hay que detenerse en el nodo anterior, porque ese es el único que puede cambiar su enlace siguiente y «saltar» al nodo eliminado. Java libera el nodo cuando ya nadie lo referencia.

Opción 05 Eliminar alumno al inicio

Retira el primer nodo y devuelve el alumno que contenía.

Parámetros. No recibe parámetros.

Retorno. Alumno: el alumno eliminado, o null si la lista estaba vacía.

Código original de eliminarInicio

```
01  public Alumno eliminarInicio() {
02      if (estaVacia()) return null;
03      Alumno eliminado = inicio.getAlumno();
04      inicio = inicio.getSiguiente();
05      cantidad--;
06      return eliminado;
07  }
```

Explicación línea por línea

Línea 01. Declara un método sin parámetros que retorna un Alumno.

Línea 02. Si la lista está vacía, devuelve null. No hay nada que retirar.

Línea 03. Guarda en eliminado la referencia al alumno del primer nodo, antes de perder el acceso a él.

Línea 04. inicio pasa a señalar al segundo nodo. El primer nodo deja de ser alcanzable desde la lista.

Línea 05. Resta uno al contador.

Líneas 06–07. Devuelve el alumno retirado y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno eliminado = lista.eliminarInicio();
```

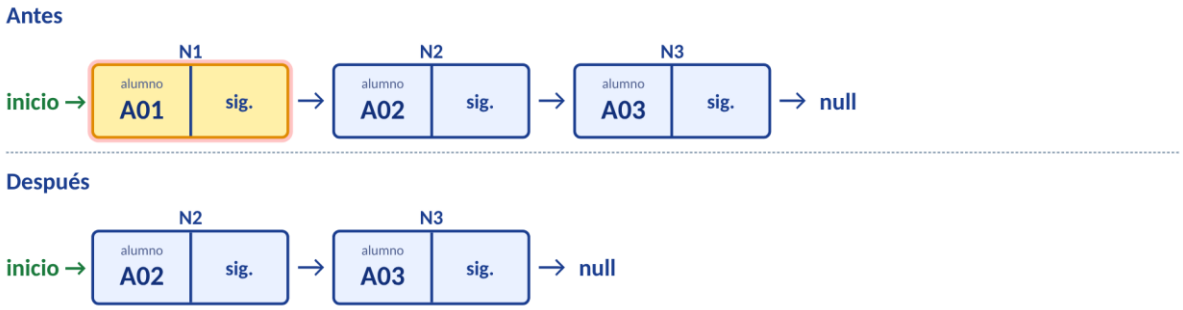


Figura 6. Eliminar al inicio. inicio avanza a N2 y N1 deja de formar parte de la lista.

Paso	Qué se hace	Qué se observa
1	Comprobar	La lista no está vacía.
2	Guardar el alumno	eliminado referencia a A01.
3	Mover inicio	inicio = N2.
4	Actualizar y devolver	cantidad pasa de 3 a 2 y se retorna A01.

Resultado comprobable. eliminado = A01; lista: A02 → A03 | cantidad = 2.

En el menú. Alumno eliminado: Codigo: A01 | Nombre: Ana | Edad: 20 | Promedio: 16.00

Casos que deben revisarse

Lista vacía. Retorna null; la Vista muestra «La lista esta vacia o no se encontro el alumno.».

Un solo nodo. inicio pasa a null y cantidad a 0.

Costo de la operación. O(1): no recorre la lista.

Opción 06 Eliminar alumno al final

Retira el último nodo y devuelve su alumno. Necesita llegar al penúltimo nodo para cortar el enlace.

Parámetros. No recibe parámetros.

Retorno. Alumno: el alumno eliminado, o null si la lista estaba vacía.

Código original de eliminarFinal

```
01  public Alumno eliminarFinal() {
02      if (estaVacia()) return null;
03      if (inicio.getSiguiente() == null) return eliminarInicio();
04      CNodo anterior = inicio;
05      while (anterior.getSiguiente().getSiguiente() != null) {
06          anterior = anterior.getSiguiente();
07      }
08      Alumno eliminado = anterior.getSiguiente().getAlumno();
09      anterior.setSiguiente(null);
10      cantidad--;
11      return eliminado;
12  }
```

Explicación línea por línea

Línea 01. Declara el método sin parámetros con retorno Alumno.

Línea 02. Con lista vacía devuelve null.

Línea 03. Si el primer nodo ya es el último, delega en eliminarInicio, que deja la lista vacía correctamente.

Línea 04. anterior comienza en el primer nodo.

Líneas 05–07. Avanza mientras el sucesor de anterior no sea el último. Al terminar, anterior es el penúltimo nodo.

Línea 08. Guarda el alumno del último nodo antes de desconectarlo.

Línea 09. El penúltimo nodo pasa a apuntar a null: ahora es el último.

Líneas 10–12. Resta uno a cantidad, devuelve el alumno retirado y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno eliminado = lista.eliminarFinal();
```

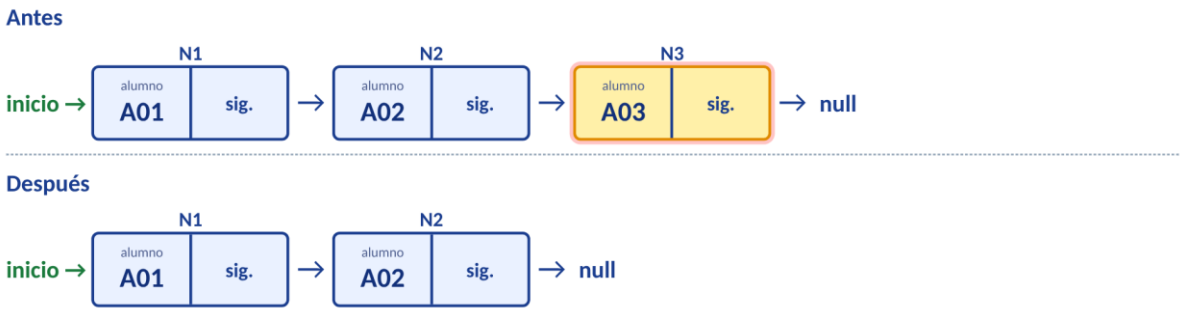


Figura 7. Eliminar al final. N2 pasa a ser el último nodo y N3 queda fuera de la cadena.

Paso	Qué se hace	Qué se observa
1	Comprobar casos especiales	La lista no está vacía y tiene más de un nodo.
2	Buscar el penúltimo	anterior = N1; N1.siguiente.siguiente (N3) no es null, avanza a N2; N2.siguiente.siguiente es null, se detiene.
3	Guardar y cortar	eliminado = A03; N2.siguiente = null.
4	Actualizar y devolver	cantidad pasa de 3 a 2.

Resultado comprobable. eliminado = A03; lista: A01 → A02 | cantidad = 2.

En el menú. Alumno eliminado: Codigo: A03 | Nombre: Sol | Edad: 19 | Promedio: 14.00

Casos que deben revisarse

Lista vacía. Retorna null.

Un solo nodo. Se delega en eliminarInicio.

Dos nodos. anterior no avanza: ya es el primero y el penúltimo.

Costo de la operación. O(n): sin puntero al penúltimo, hay que recorrer la cadena.

Opción 07 Eliminar alumno de posición X

Retira el nodo de una posición entre 1 y cantidad y devuelve su alumno.

Parámetros. posicion: 2.

Retorno. Alumno: el alumno eliminado, o null si la lista estaba vacía.

Código original de eliminarEnPosicion

```
01  public Alumno eliminarEnPosicion(int posicion) {
02      if (estaVacia()) return null;
03      if (posicion < 1 || posicion > cantidad) {
04          throw new IllegalArgumentException("Posicion valida: 1 a " + cantidad);
05      }
06      if (posicion == 1) return eliminarInicio();
07      CNodo anterior = inicio;
08      for (int i = 1; i < posicion - 1; i++) {
09          anterior = anterior.getSiguiente();
10      }
11      CNodo eliminado = anterior.getSiguiente();
12      anterior.setSiguiente(eliminado.getSiguiente());
13      cantidad--;
14      return eliminado.getAlumno();
15  }
```

Explicación línea por línea

Línea 01. Recibe la posición del nodo a retirar y devuelve el alumno eliminado.

Línea 02. Con lista vacía devuelve null antes de validar la posición.

Líneas 03–05. Rechaza posiciones menores que 1 o mayores que cantidad con una excepción que informa el rango.

Línea 06. La posición 1 se resuelve con eliminarInicio.

Línea 07. anterior comienza en el primer nodo.

Líneas 08–10. Avanza posicion − 2 veces para detenerse en el nodo previo al que se eliminará.

Línea 11. eliminado señala el nodo que está justo después de anterior.

Línea 12. anterior salta al nodo eliminado y apunta a su sucesor. Aquí se corta el nodo de la cadena.

Línea 13. Resta uno a cantidad.

Líneas 14–15. Devuelve el alumno del nodo retirado y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
```

```
02  Alumno eliminado = lista.eliminarEnPosicion(2);
```

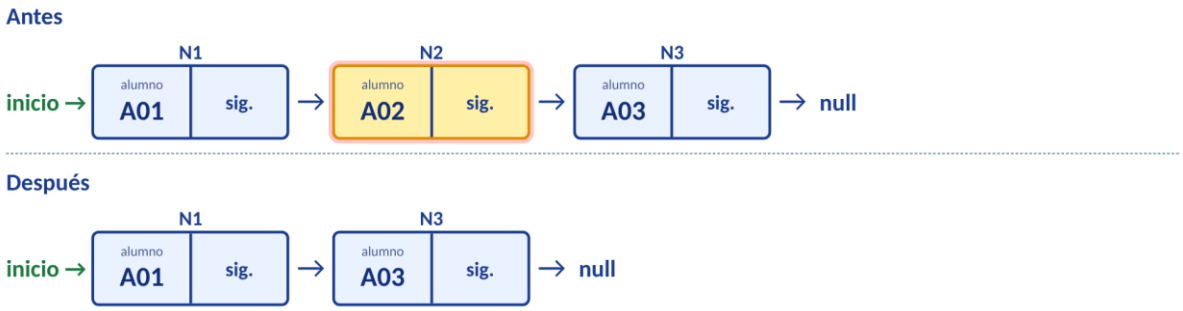


Figura 8. Eliminar la posición 2. N1 salta a N3 y N2 queda fuera de la cadena.

Paso	Qué se hace	Qué se observa
1	Validar	La lista no está vacía y 2 está entre 1 y 3.
2	Ubicar anterior	El for no se ejecuta; anterior = N1.
3	Saltar el nodo	eliminado = N2; N1.siguiente = N3.
4	Actualizar y devolver	cantidad pasa de 3 a 2 y se retorna A02.

Resultado comprobable. eliminado = A02; lista: A01 → A03 | cantidad = 2.

En el menú. Alumno eliminado: Codigo: A02 | Nombre: Luis | Edad: 21 | Promedio: 18.00

Casos que deben revisarse

Posición 1. Se delega en eliminarInicio.

Última posición. eliminado es el último nodo y anterior.siguiente pasa a null.

Posición inválida. Con 0 o 4 (lista de 3) lanza «Posicion valida: 1 a 3».

Lista vacía. Retorna null sin lanzar excepción.

Costo de la operación. O(n) en el peor caso.

Opción 08 Eliminar alumno por referencia

Retira el nodo cuyo alumno es exactamente el objeto recibido. La Vista convierte el código ingresado en esa referencia con buscarPorCodigo.

Parámetros. referencia: el objeto Alumno A02.

Retorno. Alumno: el alumno eliminado, o null si no se encontró o la lista estaba vacía.

Código original de eliminarPorReferencia

```
01  public Alumno eliminarPorReferencia(Alumno referencia) {
02      if (estaVacia() || referencia == null) return null;
03      if (inicio.getAlumno() == referencia) return eliminarInicio();
04      CNodo anterior = inicio;
05      while (anterior.getSiguiente() != null) {
06          if (anterior.getSiguiente().getAlumno() == referencia) {
07              CNodo eliminado = anterior.getSiguiente();
08              anterior.setSiguiente(eliminado.getSiguiente());
09              cantidad--;
```

```
10              return eliminado.getAlumno();
11          }
12          anterior = anterior.getSiguiente();
13      }
14      return null;
15  }
```

Explicación línea por línea

Línea 01. Recibe la referencia del alumno que se quiere retirar.

Línea 02. Con lista vacía o referencia null devuelve null. El operador || corta la evaluación en cuanto una condición es cierta.

Línea 03. Si el primer alumno es la referencia, delega en eliminarInicio.

Línea 04. anterior comienza en el primer nodo; se mirará siempre el nodo siguiente.

Línea 05. Recorre mientras exista un nodo siguiente que revisar.

Línea 06. Pregunta si el alumno del siguiente nodo es el mismo objeto de la referencia (comparación de identidad).

Líneas 07–08. Marca ese nodo como eliminado y hace que anterior apunte a su sucesor.

Líneas 09–10. Resta uno a cantidad y devuelve el alumno del nodo retirado.

Líneas 11–13. Cierran el if y, si no hubo coincidencia, anterior avanza al siguiente nodo.

Líneas 14–15. Si el recorrido termina sin hallar la referencia, devuelve null y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno ref = lista.buscarPorCodigo("A02");
03  Alumno eliminado = lista.eliminarPorReferencia(ref);
```

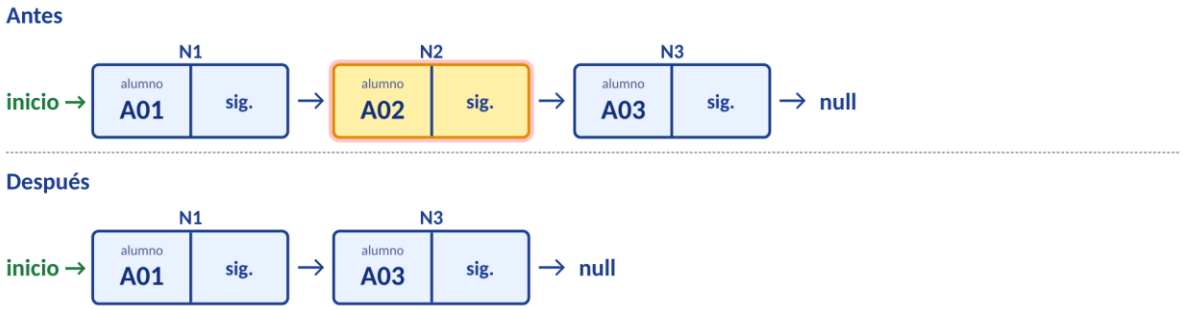


Figura 9. Eliminar por referencia. El nodo de A02 queda fuera de la cadena.

Paso	Qué se hace	Qué se observa
1	Conseguir la referencia	La Vista obtiene con buscarPorCodigo el objeto que está en N2.
2	Descartar el primero	El alumno de N1 no es la referencia.

Paso	Qué se hace	Qué se observa
3	Buscar con anterior	anterior = N1 y N1.siguiente guarda el alumno de N2: coincide.
4	Saltar y devolver	N1.siguiente = N3; cantidad pasa de 3 a 2; se retorna A02.

Resultado comprobable. eliminado = A02; lista: A01 → A03 | cantidad = 2.

En el menú. Alumno eliminado: Codigo: A02 | Nombre: Luis | Edad: 21 | Promedio: 18.00

Casos que deben revisarse

Referencia nula. Si el código no existe, la Vista pasa null y el método retorna null.

Objeto ajeno a la lista. Recorre todos los nodos y retorna null.

Primer alumno. Se resuelve con eliminarInicio.

Costo de la operación. O(n) en el peor caso.

9 Búsqueda y recorrido

Estas dos operaciones leen la lista sin cambiar enlaces, datos ni contador. Ambas recorren la cadena con una variable local actual que avanza por siguiente hasta encontrar lo buscado o llegar a null.

Opción 09 Buscar alumno por código

Busca el alumno cuyo código coincide con el texto recibido, sin distinguir mayúsculas.

Parámetros. codigo: " a02 " (con espacios y minúsculas).

Retorno. Alumno: la referencia al alumno encontrado, o null si no existe.

Código original de buscarPorCodigo

```
01  public Alumno buscarPorCodigo(String codigo) {
02      if (codigo == null) return null;
03      CNodo actual = inicio;
04      while (actual != null) {
05          if (actual.getAlumno().getCodigo().equalsIgnoreCase(codigo.trim())) {
06              return actual.getAlumno();
07          }
08          actual = actual.getSiguiente();
09      }
10      return null;
11  }
```

Explicación línea por línea

Línea 01. Recibe un texto con el código buscado y devuelve un Alumno.

Línea 02. Con un código null devuelve null sin recorrer la lista.

Línea 03. actual comienza en el primer nodo.

Línea 04. Repite mientras actual señale un nodo real.

Línea 05. Compara el código del alumno con codigo.trim(), ignorando mayúsculas y minúsculas.

Líneas 06–07. Si coincide, devuelve la referencia al mismo objeto almacenado en el nodo.

Línea 08. Si no coincide, avanza al nodo siguiente.

Líneas 09–11. Cierra el recorrido; si terminó sin coincidencia, devuelve null.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  Alumno r = lista.buscarPorCodigo(" a02 ");
03  System.out.println(r);
```

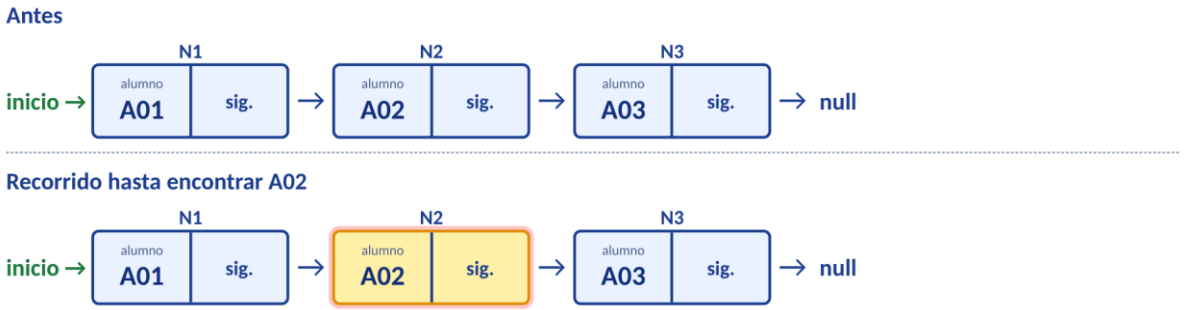


Figura 10. Búsqueda del código A02. El recorrido se detiene en N2.

Paso	Qué se hace	Qué se observa
1	Preparar el texto	codigo.trim() produce «a02».
2	Comparar N1	«A01» no coincide con «a02».
3	Comparar N2	equalsIgnoreCase(«A02», «a02») es true.
4	Devolver	Se retorna la referencia al Alumno de N2; la lista no cambia.

Resultado comprobable. Codigo: A02 | Nombre: Luis | Edad: 21 | Promedio: 18.00.

En el menú. Se imprime el alumno encontrado; si el resultado es null, «Alumno no encontrado.».

Casos que deben revisarse

Código nulo. Devuelve null.

Sin coincidencia. Recorre todos los nodos y devuelve null.

Mejor y peor caso. O(1) si está en el primero; O(n) si está al final o no existe.

Uso interno. Lo usan validarAlumnoNuevo, modificar y la Vista para obtener referencias.

Opción 10 Mostrar todos los alumnos

Recorre la lista completa y construye un texto numerado con los datos de cada alumno.

Parámetros. No recibe parámetros.

Retorno. String: el texto con todos los alumnos, o un aviso si la lista está vacía.

Código original de mostrarTodos

```
01  public String mostrarTodos() {
02      if (estaVacia()) return "No hay alumnos registrados.";
03      StringBuilder texto = new StringBuilder();
04      CNodo actual = inicio;
05      int posicion = 1;
06      while (actual != null) {
07          texto.append(posicion).append(". ").append(actual.getAlumno())
08                  .append(System.lineSeparator());
09          actual = actual.getSiguiente();
10          posicion++;
11      }
12      return texto.toString();
13  }
```

Explicación línea por línea

Línea 01. Declara un método que devuelve texto.

Línea 02. Con lista vacía devuelve un mensaje informativo.

Línea 03. Crea un StringBuilder para ir acumulando las líneas sin generar muchas cadenas intermedias.

Líneas 04–05. actual comienza en el primer nodo y posicion empieza en 1.

Línea 06. Repite mientras haya nodos por visitar.

Líneas 07–08. Agrega número, un punto, el alumno (se convierte con toString) y un salto de línea.

Líneas 09–10. Avanza al siguiente nodo y suma uno a la posición.

Líneas 11–13. Cierra el recorrido, devuelve el texto completo y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  System.out.println(lista.mostrarTodos());
```

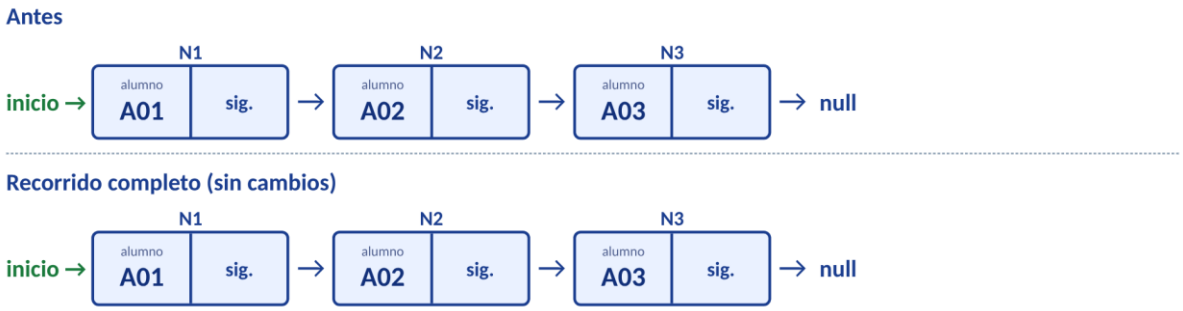


Figura 11. Recorrido completo: se visitan N1, N2 y N3 sin modificar la cadena.

Paso	Qué se hace	Qué se observa
1	Comprobar	La lista no está vacía.
2	Preparar	actual = N1, posicion = 1.
3	Recorrer	Se agregan las líneas de A01, A02 y A03 con posiciones 1, 2 y 3.
4	Terminar	actual llega a null y se retorna el texto.

Resultado comprobable. 1. Codigo: A01 | Nombre: Ana | Edad: 20 | Promedio: 16.00; 2. Codigo: A02 | Nombre: Luis | …; 3. Codigo: A03 | Nombre: Sol | ….

En el menú. Se imprime una línea en blanco y luego la lista numerada.

Casos que deben revisarse

Lista vacía. Retorna «No hay alumnos registrados.».

Sin cambios. No altera enlaces, datos ni cantidad.

Costo de la operación. O(n): visita cada nodo una vez.

10 Ordenamiento de alumnos

Las opciones 11 y 12 llaman al mismo método ordenar, que aplica el algoritmo de la burbuja: compara pares de nodos vecinos y, si están en el orden contrario, intercambia los alumnos que guardan. Los enlaces no cambian: los nodos quedan donde estaban y lo que se mueve son los datos.

10.1 Código compartido de las opciones 11 y 12

Código original de ordenar

```
01  public void ordenar(boolean porPromedio) {
02      for (int i = 0; i < cantidad - 1; i++) {
03          CNodo actual = inicio;
04          while (actual.getSiguiente() != null) {
05              Alumno a = actual.getAlumno();
06              Alumno b = actual.getSiguiente().getAlumno();
07              boolean cambiar = porPromedio
08                      ? a.getPromedio() < b.getPromedio()
09                      : a.getCodigo().compareToIgnoreCase(b.getCodigo()) > 0;
10              if (cambiar) {
11                  actual.setAlumno(b);
12                  actual.getSiguiente().setAlumno(a);
13              }
14              actual = actual.getSiguiente();
15          }
16      }
17  }
```

Explicación línea por línea

Línea 01. El parámetro porPromedio elige el criterio: true ordena por promedio y false por código.

Línea 02. El ciclo externo repite cantidad − 1 pasadas. Con 0 o 1 nodos no se ejecuta.

Línea 03. Cada pasada comienza en el primer nodo.

Línea 04. Mientras el nodo actual tenga un siguiente, se compara el par actual–siguiente.

Líneas 05–06. a es el alumno del nodo actual y b el del nodo siguiente.

Líneas 07–09. El operador condicional decide si hay que cambiar. Por promedio: a.promedio < b.promedio (orden descendente). Por código: a.codigo va después que b.codigo (orden ascendente, sin distinguir mayúsculas).

Líneas 10–13. Si hay que cambiar, el nodo actual guarda a b y el nodo siguiente guarda a a. Solo se intercambian los datos.

Líneas 14–15. Avanza al nodo siguiente y cierra el ciclo interno.

Líneas 16–17. Cierra el ciclo externo y el método.

Cada pasada deja el elemento que corresponde al final de la zona revisada. El costo es $O(n^2)$ en comparaciones, porque hay del orden de n pasadas y cada una recorre del orden de n nodos. Cuando dos promedios son iguales no se intercambian (el operador < es estricto), por lo que esos alumnos conservan su orden relativo.

Opción 11 Ordenar alumnos por código

Ordena la lista de menor a mayor código llamando a ordenar(false).

Parámetros. No recibe parámetros.

Retorno. void: no devuelve un valor.

Código original de ordenarPorCodigo

```
01  public void ordenarPorCodigo() {
02      ordenar(false);
03  }
```

Prueba de escritorio y resultado

Se usa una lista que inicia desordenada: N1 guarda A03, N2 guarda A01 y N3 guarda A02.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaParaOrdenar();
02  lista.ordenarPorCodigo();
```

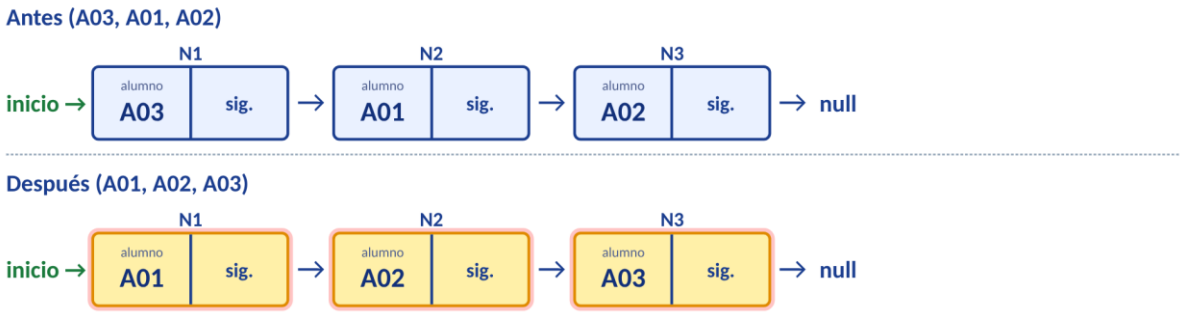


Figura 12. Ordenar por código. Los nodos conservan sus enlaces; cambian los alumnos que guardan.

Paso	Qué se hace	Qué se observa
1	Pasada 1, par N1–N2	A03 va después de A01: se intercambian. Queda A01, A03, A02.
2	Pasada 1, par N2–N3	A03 va después de A02: se intercambian. Queda A01, A02, A03.
3	Pasada 2, par N1–N2	A01 no va después de A02: no cambia.
4	Pasada 2, par N2–N3	A02 no va después de A03: no cambia.

Resultado comprobable. A01 → A02 → A03 | cantidad = 3.

En el menú. Ordenado por codigo (A-Z). y a continuación la lista.

Casos que deben revisarse

Lista vacía o de un nodo. cantidad − 1 no es positivo; el ciclo externo no se ejecuta y no hay error.

Ya ordenada. Hace las comparaciones pero no intercambia.

Mayúsculas. compareToIgnoreCase ignora la diferencia entre mayúsculas y minúsculas.

Costo de la operación. $O(n^2)$.

Opción 12 Ordenar alumnos por promedio

Ordena la lista de mayor a menor promedio llamando a ordenar(true).

Parámetros. No recibe parámetros.

Retorno. void: no devuelve un valor.

Código original de ordenarPorPromedio

```
01  public void ordenarPorPromedio() {
02      ordenar(true);
03  }
```

Prueba de escritorio y resultado

Se parte de la lista base: A01 tiene 16.0, A02 tiene 18.0 y A03 tiene 14.0.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  lista.ordenarPorPromedio();
```

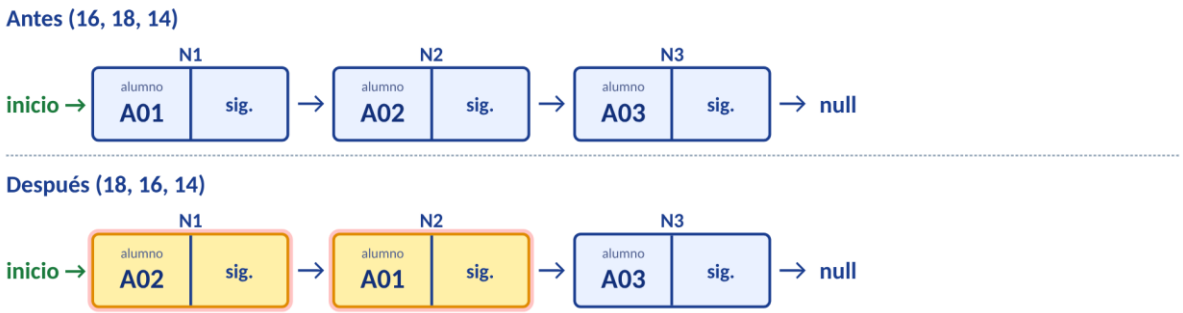


Figura 13. Ordenar por promedio. A02 (18.0) pasa al primer nodo.

Paso	Qué se hace	Qué se observa
1	Pasada 1, par N1–N2	16.0 < 18.0: se intercambian. Queda A02, A01, A03.
2	Pasada 1, par N2–N3	16.0 < 14.0 es falso: no cambia.
3	Pasada 2, par N1–N2	18.0 < 16.0 es falso: no cambia.
4	Pasada 2, par N2–N3	16.0 < 14.0 es falso: no cambia.

Resultado comprobable. A02 (18.0) → A01 (16.0) → A03 (14.0) | cantidad = 3.

En el menú. Ordenado por promedio (mayor a menor). y a continuación la lista.

Casos que deben revisarse

Promedios iguales. No se intercambian; conservan su orden relativo.

Lista vacía o de un nodo. No se ejecutan las pasadas.

Comparación. Se comparan promedios double con el operador <.

Costo de la operación. $O(n^2)$.

11 Intercambio de alumnos

Opción 13 Intercambiar dos alumnos

Intercambia los alumnos guardados en los nodos que corresponden a dos códigos. Los nodos no se mueven: se permutan los datos.

Parámetros. codigo1: «A01»; codigo2: «A03».

Retorno. boolean: true si se intercambiaron; false si algún código no existe.

Código original de intercambiar

```
01  public boolean intercambiar(String codigo1, String codigo2) {
02      CNodo primero = buscarNodo(codigo1);
03      CNodo segundo = buscarNodo(codigo2);
04      if (primero == null || segundo == null) return false;
05      Alumno temporal = primero.getAlumno();
06      primero.setAlumno(segundo.getAlumno());
07      segundo.setAlumno(temporal);
08      return true;
09  }
```

Explicación línea por línea

Línea 01. Recibe dos códigos y devuelve true o false.

Líneas 02–03. Busca con el auxiliar buscarNodo el nodo de cada código. El resultado es un CNodo, no un Alumno.

Línea 04. Si falta alguno de los dos nodos, devuelve false y no cambia nada.

Línea 05. Guarda en temporal el alumno del primer nodo para no perderlo.

Línea 06. El primer nodo pasa a guardar el alumno del segundo.

Línea 07. El segundo nodo guarda el alumno que estaba en temporal.

Líneas 08–09. Devuelve true y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  boolean ok = lista.intercambiar("A01", "A03");
```

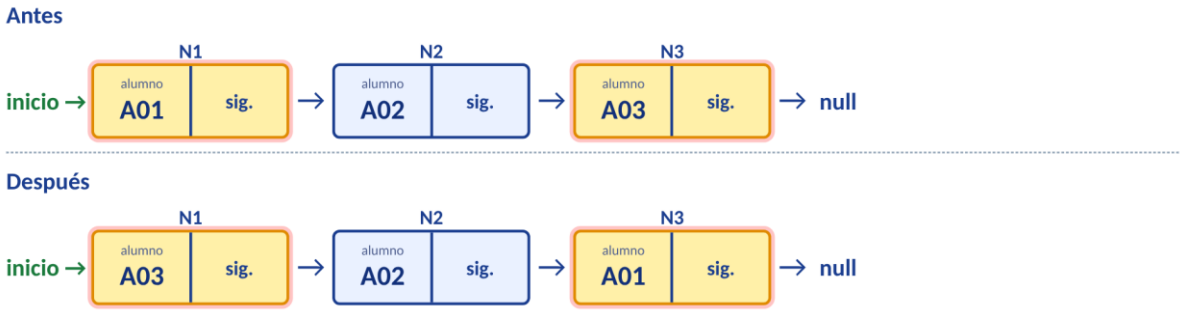


Figura 14. Intercambio de A01 y A03. Los nodos N1 y N3 conservan sus enlaces.

Paso	Qué se hace	Qué se observa
1	Buscar nodos	primero = N1 y segundo = N3.
2	Guardar temporal	temporal referencia a A01.
3	Primer cambio	N1 guarda a A03.
4	Segundo cambio	N3 guarda a temporal (A01) y se retorna true.

Resultado comprobable. ok = true; lista: A03 → A02 → A01 | cantidad = 3.

En el menú. Alumnos intercambiados.

Casos que deben revisarse

Código inexistente. Retorna false; la Vista muestra «Uno o ambos codigos no existen.».

Mismo código dos veces. Ambos nodos son el mismo; el resultado es true y la lista queda igual.

Datos y enlaces. Se cambian datos, no enlaces; por eso no hace falta tocar nodos anteriores.

Costo de la operación. O(n) por las dos búsquedas.

12 Tamaño, modificación y estado de la lista

Opción 14 Mostrar cantidad de alumnos

Devuelve cuántos alumnos hay en la lista leyendo el contador cantidad.

Parámetros. No recibe parámetros.

Retorno. int: el número de alumnos.

Código original de obtenerTamanio

```
01  public int obtenerTamanio() {
02      return cantidad;
03  }
```

Explicación línea por línea

Línea 01. Declara un método sin parámetros que retorna un entero.

Línea 02. Devuelve el valor de cantidad; no recorre la lista.

Línea 03. Cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  int n = lista.obtenerTamanio();
```

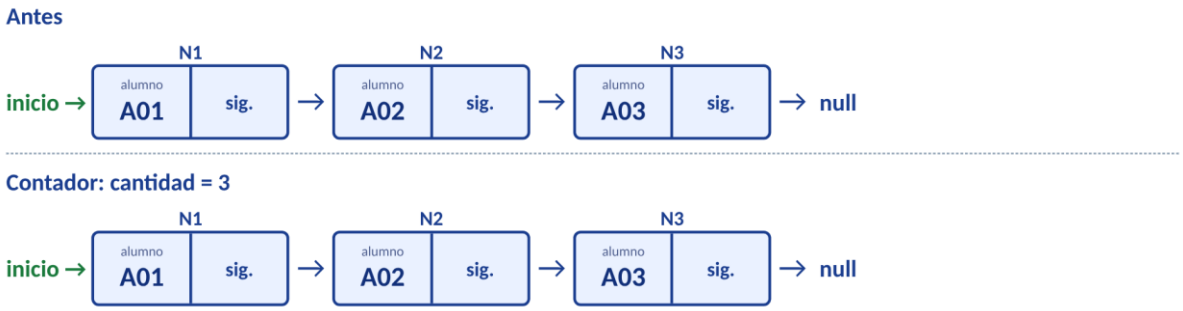


Figura 15. La cantidad se lee del contador, no se calcula recorriendo los nodos.

Paso	Qué se hace	Qué se observa
1	Leer el contador	cantidad vale 3 por las tres inserciones de crearListaBase.
2	Devolver	Se retorna 3.

Resultado comprobable. n = 3.

En el menú. Cantidad de alumnos: 3

Casos que deben revisarse

Lista vacía. Retorna 0.

Sincronía. Cada inserción suma uno y cada eliminación resta uno, por lo que el contador coincide con el número de nodos.

Costo de la operación. O(1).

Opción 15 Modificar datos de un alumno

Cambia nombre, edad y promedio del alumno con el código indicado. El código se conserva.

Parámetros. codigo: «A02»; nombre: «Luis Fernando»; edad: 22; promedio: 19.0.

Retorno. boolean: true si lo modificó; false si el código no existe.

Código original de modificar

```
01  public boolean modificar(String codigo, String nombre, int edad, double promedio) {
02      Alumno alumno = buscarPorCodigo(codigo);
03      if (alumno == null) return false;
04      // Validar todos los datos ANTES de modificar algún atributo.
05      Alumno datosValidos = new Alumno(alumno.getCodigo(), nombre, edad, promedio);
06      alumno.setNombre(datosValidos.getNombre());
07      alumno.setEdad(datosValidos.getEdad());
08      alumno.setPromedio(datosValidos.getPromedio());
09      return true;
10  }
```

Explicación línea por línea

Línea 01. Recibe el código del alumno y los tres datos nuevos.

Línea 02. Obtiene la referencia al alumno almacenado con ese código.

Línea 03. Si no existe, devuelve false.

Línea 04. Comentario del código: los datos se validan antes de cambiar cualquier atributo.

Línea 05. Construye un Alumno temporal con el mismo código y los datos nuevos. Si algún dato es inválido, el constructor lanza una excepción y el alumno original queda intacto.

Líneas 06–08. Copia los tres datos validados al alumno original mediante sus setters.

Líneas 09–10. Devuelve true y cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE lista = crearListaBase();
02  boolean ok = lista.modificar("A02", "Luis Fernando", 22, 19.0);
```

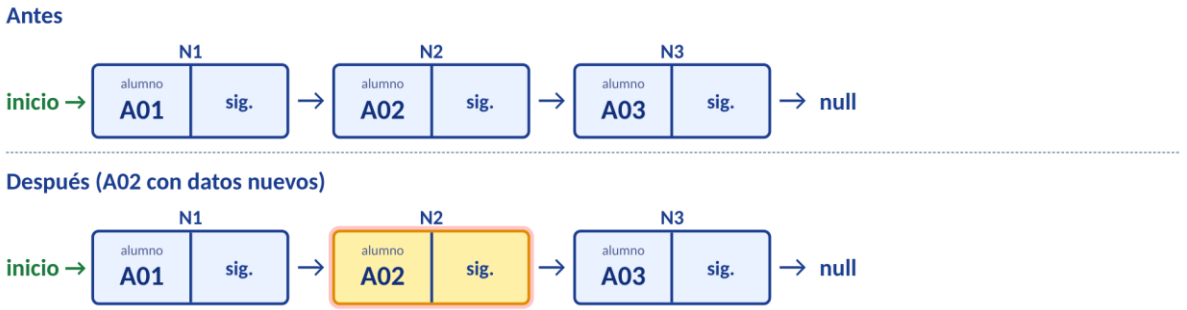


Figura 16. Modificación de A02. El nodo y los enlaces no cambian; cambian los datos del alumno.

Paso	Qué se hace	Qué se observa
1	Buscar	buscarPorCodigo("A02") devuelve el alumno de N2.
2	Validar	Se crea el Alumno temporal con los datos nuevos; todos son válidos.
3	Copiar	Nombre, edad y promedio del alumno de N2 pasan a «Luis Fernando», 22 y 19.0.
4	Devolver	Se retorna true.

Resultado comprobable. ok = true; A02 ahora es Luis Fernando · 22 · 19.00. El orden y la cantidad no cambian.

En el menú. Alumno modificado. Se conserva su codigo.

Casos que deben revisarse

Código inexistente. Retorna false; la Vista avisa «Alumno no encontrado.» antes de pedir los datos.

Dato inválido. Por ejemplo un promedio de 25: la excepción ocurre en la línea 05, antes de cambiar nada.

Código inmutable. El código no se modifica porque Alumno no tiene setCodigo.

Costo de la operación. O(n) por la búsqueda.

Opción 16 Verificar si la lista está vacía

Indica si la lista no tiene nodos.

Parámetros. No recibe parámetros.

Retorno. boolean: true si inicio es null; false en caso contrario.

Código original de estaVacia

```
01  public boolean estaVacia() {
02      return inicio == null;
03  }
```

Explicación línea por línea

Línea 01. Declara un método sin parámetros que retorna true o false.

Línea 02. La expresión inicio == null ya es un valor booleano; se devuelve directamente.

Línea 03. Cierra el método.

Prueba de escritorio y resultado

Cada prueba es independiente: se parte de una lista nueva y los cambios de otra opción no se arrastran a este ejemplo.

Fragmento didáctico para probar la operación

```
01  CListaSE vacia = new CListaSE();
02  CListaSE base = crearListaBase();
03  boolean a = vacia.estaVacia();
04  boolean b = base.estaVacia();
```

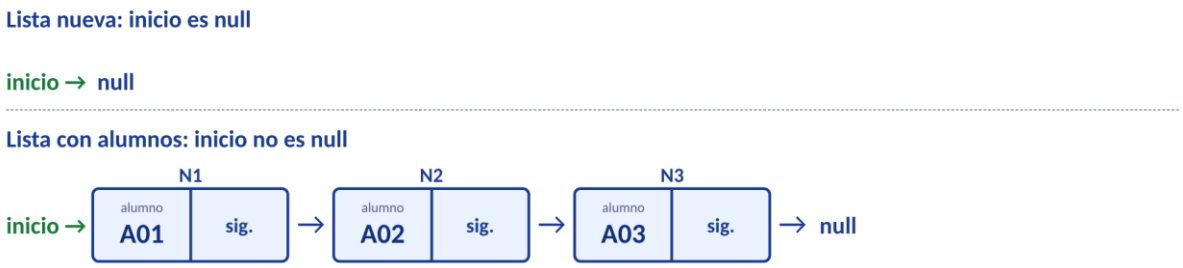


Figura 17. Una lista nueva tiene inicio en null; una lista con alumnos apunta a su primer nodo.

Paso	Qué se hace	Qué se observa
1	Lista nueva	inicio es null: estaVacia devuelve true.
2	Lista base	inicio señala a N1: estaVacia devuelve false.

Resultado comprobable. a = true; b = false.

En el menú. La lista esta vacia: false

Casos que deben revisarse

Después de eliminar todo. Al retirar el último nodo, inicio vuelve a null y el método devuelve true.

Uso interno. insertarFinal, eliminarInicio, eliminarFinal, eliminarEnPosicion, eliminarPorReferencia y mostrarTodos lo usan.

Costo de la operación. O(1).

13 Funcionamiento de la Vista

13.1 Repetición del menú y manejo de avisos

El método main usa un ciclo do-while: muestra el menú, lee la opción con leerEntero y llama a ejecutar dentro de un bloque try. Si CListaSE o Alumno lanzan una IllegalArgumentException, el catch imprime «AVISO:» seguido del mensaje y el programa continúa con el siguiente ciclo. El ciclo termina cuando la opción es 0.

El método ejecutar usa un switch con un caso por opción. Cada caso pide los datos necesarios, llama al método de la lista y muestra un mensaje de confirmación o el resultado. El caso default imprime «Opcion no valida.».

13.2 Apertura y ejecución del proyecto

1. Abrir la carpeta CSemana10 como proyecto en NetBeans (proyecto Ant con paquetes Modelo, DAO y Vista).
2. Verificar en las propiedades del proyecto que la clase principal sea Vista.Principal. El proyecto está configurado con codificación UTF-8 y compilación para Java 22.
3. Ejecutar el proyecto (Run) y trabajar desde la consola de salida de NetBeans.
4. Elegir las opciones por número. Después de cada operación, usar la opción 10 para ver la lista y la 14 para comprobar la cantidad.

13.3 Responsabilidad de cada auxiliar de entrada

Método	Qué hace	Si el dato es inválido
pedirTexto	Muestra el mensaje, lee una línea y quita espacios exteriores.	No valida; devuelve el texto leído.
leerEntero	Lee la opción del menú y la convierte a int.	Muestra «Debe ingresar un numero entero.» y devuelve −1, que cae en «Opcion no valida.».
pedirEntero	Pide un texto y lo convierte a int (posición o edad).	Lanza IllegalArgumentException; el catch de main muestra el aviso.
pedirDecimal	Pide un texto, cambia la coma por punto y lo convierte a double.	Lanza IllegalArgumentException «Debe ingresar un numero decimal valido.».
leerAlumno	Pide código, nombre, edad y promedio y construye un Alumno.	El constructor de Alumno lanza la excepción que corresponda.
mostrarEliminado	Imprime el alumno eliminado o un aviso si el resultado es null.	No aplica.

13.4 Condiciones de la Vista que ayudan al caso

- En la opción 4, la Vista busca la referencia con buscarPorCodigo y, si no existe, muestra «El codigo de referencia no existe.» sin llamar a la lista.
- En la opción 8, la Vista obtiene la referencia con buscarPorCodigo y delega en eliminarPorReferencia; si el código no existe, la referencia es null y el resultado es null.
- En la opción 15, la Vista comprueba que el código exista antes de pedir los datos nuevos.

- Las opciones 9 y 13 usan un operador condicional para elegir entre dos mensajes según el resultado.
- Como pedirEntero y pedirDecimal lanzan IllegalArgumentException, un solo catch en main cubre los errores de lectura y los de la lista.

13.5 Ejemplos de entradas inválidas

Entrada	Dónde se detecta	Mensaje
Opción «abc»	leerEntero	Debe ingresar un numero entero. y luego Opcion no valida.
Edad «veinte»	pedirEntero	AVISO: Debe ingresar un numero entero.
Promedio «25»	Alumno.setPromedio	AVISO: El promedio debe estar entre 0 y 20.
Código repetido	validarAlumnoNuevo	AVISO: Ya existe un alumno con ese codigo.
Posición 9 (lista de 3)	insertarEnPosicion	AVISO: Posicion valida: 1 a 4
Código «Z99» en la opción 15	Principal (caso 15)	Alumno no encontrado.

14 Laboratorio guiado

14.1 Práctica integrada con todas las operaciones

La práctica recorre las operaciones del menú sobre una única lista que comienza vacía. En cada paso se indica el estado esperado, de modo que el estudiante pueda hacer la prueba de escritorio y luego comparar con la salida del programa.

Paso	Operación	Estado esperado de la lista	cantidad
1	insertarFinal(A01 Ana 20 16.0)	A01	1
2	insertarFinal(A02 Luis 21 18.0)	A01 → A02	2
3	insertarFinal(A03 Sol 19 14.0)	A01 → A02 → A03	3
4	insertarInicio(A00 Mía 18 15.0)	A00 → A01 → A02 → A03	4
5	insertarEnPosicion(A04 Iván 22 17.0, 3)	A00 → A01 → A04 → A02 → A03	5
6	insertarPorReferencia(A02, A05 Lucía 20 19.0)	A00 → A01 → A04 → A02 → A05 → A03	6
7	buscarPorCodigo("a04")	Devuelve Iván; la lista no cambia.	6
8	ordenarPorPromedio()	A05 (19) → A02 (18) → A04 (17) → A01 (16) → A00 (15) → A03 (14)	6
9	ordenarPorCodigo()	A00 → A01 → A02 → A03 → A04 → A05	6
10	intercambiar("A00", "A05")	A05 → A01 → A02 → A03 → A04 → A00	6
11	modificar("A03", "Sol María", 20, 15.5)	Mismo orden; A03 con los datos nuevos.	6
12	eliminarInicio()	Retira A05: A01 → A02 → A03 → A04 → A00	5
13	eliminarFinal()	Retira A00: A01 → A02 → A03 → A04	4
14	eliminarEnPosicion(2)	Retira A02: A01 → A03 → A04	3
15	eliminarPorReferencia(A04)	Retira A04: A01 → A03	2
16	estaVacia() y obtenerTamanio()	false y 2.	2

14.2 Código didáctico de la práctica integrada

Este fragmento no forma parte del proyecto original: es apoyo para comprobar la tabla anterior desde un método main de una clase de pruebas.

Programa de prueba

```
01  CListaSE lista = new CListaSE();
02  lista.insertarFinal(new Alumno("A01", "Ana", 20, 16.0));
03  lista.insertarFinal(new Alumno("A02", "Luis", 21, 18.0));
04  lista.insertarFinal(new Alumno("A03", "Sol", 19, 14.0));
05  lista.insertarInicio(new Alumno("A00", "Mía", 18, 15.0));
06  lista.insertarEnPosicion(new Alumno("A04", "Iván", 22, 17.0), 3);
07  Alumno ref = lista.buscarPorCodigo("A02");
08  lista.insertarPorReferencia(ref, new Alumno("A05", "Lucía", 20, 19.0));
09  System.out.println(lista.buscarPorCodigo("a04"));
10  lista.ordenarPorPromedio();
```

11	System.out.println(lista.mostrarTodos());
12	lista.ordenarPorCodigo();
13	lista.intercambiar("A00", "A05");
14	lista.modificar("A03", "Sol María", 20, 15.5);
15	System.out.println(lista.mostrarTodos());
16	lista.eliminarInicio();
17	lista.eliminarFinal();
18	lista.eliminarEnPosicion(2);
19	lista.eliminarPorReferencia(lista.buscarPorCodigo("A04"));
20	System.out.println(lista.mostrarTodos());
21	System.out.println(lista.estaVacia() + "	" + lista.obtenerTamanio());

14.3 Casos límite que se deben justificar

Caso	Qué ocurre	Por qué
Eliminar de una lista vacía	eliminarInicio, eliminarFinal, eliminarEnPosicion y eliminarPorReferencia devuelven null.	Todas comprueban estaVacia() antes de usar inicio.
Eliminar el único nodo	inicio queda en null y cantidad en 0.	eliminarFinal delega en eliminarInicio cuando hay un solo nodo.
Insertar en cantidad + 1	Equivale a insertar al final.	El for avanza hasta el último nodo y su siguiente es null.
Código repetido	La inserción lanza excepción y no cambia la lista.	validarAlumnoNuevo se ejecuta antes de crear el nodo.
Referencia equivalente pero distinta	No se encuentra en la lista.	La comparación es por identidad (==), no por contenido.
Ordenar con un solo alumno	No cambia nada.	cantidad − 1 vale 0 y el ciclo externo no se ejecuta.
Modificar con dato inválido	Lanza excepción y deja intacto al alumno.	Se valida con un Alumno temporal antes de copiar los datos.

15 Repaso y conclusiones

15.1 Comparación de las operaciones

Opción	Método	Qué cambia	Costo
1	insertarInicio	inicio, un enlace nuevo y cantidad	O(n) por validar; enlazar O(1)
2	insertarFinal	siguiente del último y cantidad	O(n)
3	insertarEnPosicion	siguiente del anterior y cantidad	O(n)
4	insertarPorReferencia	siguiente del nodo de la referencia y cantidad	O(n)
5	eliminarInicio	inicio y cantidad	O(1)
6	eliminarFinal	siguiente del penúltimo y cantidad	O(n)
7	eliminarEnPosicion	siguiente del anterior y cantidad	O(n)
8	eliminarPorReferencia	siguiente del anterior y cantidad	O(n)
9	buscarPorCodigo	Nada (consulta)	O(n); mejor caso O(1)
10	mostrarTodos	Nada (consulta)	O(n)
11 y 12	ordenar(false) y ordenar(true)	Los alumnos guardados en los nodos	$O(n^2)$
13	intercambiar	Los alumnos de dos nodos	O(n)
14	obtenerTamanio	Nada (consulta)	O(1)
15	modificar	Datos del alumno	O(n)
16	estaVacia	Nada (consulta)	O(1)

15.2 Preguntas de repaso con respuesta

1. ¿Qué diferencia hay entre un Alumno y un CNodo? Alumno guarda los datos de la persona; CNodo guarda la referencia a ese Alumno y la referencia al siguiente nodo. La lista se forma con nodos, no con alumnos sueltos.

2. ¿Por qué insertarInicio no es O(1) en este proyecto? Enlazar al principio solo cambia dos referencias, pero antes validarAlumnoNuevo busca si el código ya existe y recorre la lista.

3. ¿Por qué en eliminarEnPosicion se necesita el nodo anterior? Porque solo el nodo anterior puede cambiar su enlace siguiente para saltar al nodo eliminado. Un nodo no conoce a su predecesor.

4. ¿Qué pasa si se invierten las dos asignaciones de enlace al insertar en el medio? Si anterior.siguiente apunta al nuevo antes de que el nuevo apunte al sucesor, se pierde el acceso al resto de la cadena.

5. ¿Por qué ordenar e intercambiar no cambian enlaces? Porque usan setAlumno para permutar los datos que guardan los nodos. Es más simple que reconectar nodos y deja la estructura intacta.

6. ¿Por qué modificar crea un Alumno temporal? Para validar nombre, edad y promedio antes de tocar al alumno real. Si algún dato es inválido, la excepción impide que el alumno quede a medio cambiar.

15.3 Conclusiones del caso

1. La lista simplemente enlazada resuelve las operaciones del enunciado sin arreglos ni ArrayList: toda la estructura está en las referencias siguiente.
2. La clave de las inserciones y eliminaciones es el orden de las asignaciones de enlace y el uso del nodo anterior.
3. El proyecto separa responsabilidades: Modelo guarda y valida datos, DAO administra la cadena y Vista interactúa con el usuario.
4. Las validaciones centralizadas (Alumno y validarAlumnoNuevo) evitan estados inválidos: códigos repetidos, promedios fuera de rango o alumnos nulos.
5. Distinguir entre cambiar enlaces (insertar, eliminar) y cambiar datos (ordenar, intercambiar, modificar) permite predecir qué nodos se ven afectados en cada operación.
6. Como la lista no guarda un puntero al último nodo ni al anterior, varias operaciones tienen costo lineal; ese es el precio de la estructura simple.

16 Código completo de la Vista

Se incluye el código original de las dos clases de la Vista para que el informe sea autocontenido.

16.1 MenuOpciones

Código original de Vista.MenuOpciones

```
01  package Vista;
02
03  public class MenuOpciones {
04      public static void mostrar() {
05          System.out.println("\n================================================");
06          System.out.println("       SISTEMA DE GESTION DE ALUMNOS");
07          System.out.println("         LISTA SIMPLEMENTE ENLAZADA");
08          System.out.println("================================================");
09          System.out.println("       INSERCION           ");
10          System.out.println("  1. Insertar alumno al inicio");
11          System.out.println("  2. Insertar alumno al final");
12          System.out.println("  3. Insertar alumno en posicion X");
13          System.out.println("  4. Insertar alumno por referencia (despues)");
14          System.out.println("       ELIMINACION           ");
15          System.out.println("  5. Eliminar alumno al inicio");
16          System.out.println("  6. Eliminar alumno al final");
17          System.out.println("  7. Eliminar alumno de posicion X");
18          System.out.println("  8. Eliminar alumno por referencia / codigo");
19          System.out.println(" ----------- ");
20          System.out.println("  9. Buscar alumno por codigo");
21          System.out.println(" 10. Mostrar todos los alumnos");
22          System.out.println(" 11. Ordenar alumnos por codigo (ascendente)");
23          System.out.println(" 12. Ordenar alumnos por promedio (descendente)");
24          System.out.println(" 13. Intercambiar dos alumnos");
25          System.out.println(" 14. Mostrar cantidad de alumnos");
26          System.out.println(" 15. Modificar datos de un alumno");
27          System.out.println(" 16. Verificar si la lista esta vacia");
28          System.out.println("  0. Salir del sistema");
29          System.out.println("================================================");
30          System.out.print("Seleccione una opcion: ");
31      }
32  }
```

16.2 Principal

Código original de Vista.Principal

```
01  package Vista;
02
03  import DAO.CListaSE;
```

```
04  import Modelo.Alumno;
05  import java.util.Scanner;
06
07  public class Principal {
08      private static final Scanner teclado = new Scanner(System.in);
09      private static final CListaSE lista = new CListaSE();
10
11      public static void main(String[] args) {
12          int opcion;
13          do {
14              MenuOpciones.mostrar();
15              opcion = leerEntero();
16              try {
17                  ejecutar(opcion);
18              } catch (IllegalArgumentException ex) {
19                  System.out.println("AVISO: " + ex.getMessage());
20              }
21          } while (opcion != 0);
22          // No se cierra System.in durante la ejecucion del menu.
23      }
24
25      private static void ejecutar(int opcion) {
26          switch (opcion) {
27              case 1:
28                  lista.insertarInicio(leerAlumno());
29                  System.out.println("Alumno insertado al inicio.");
30                  break;
31              case 2:
32                  lista.insertarFinal(leerAlumno());
33                  System.out.println("Alumno insertado al final.");
34                  break;
35              case 3:
36                  int posicionInsertar = pedirEntero("Posicion (desde 1): ");
37                  lista.insertarEnPosicion(leerAlumno(), posicionInsertar);
38                  System.out.println("Alumno insertado en posicion " + posicionInsertar);
39                  break;
40              case 4:
41                  String codRefIns = pedirTexto("Codigo del alumno de referencia: ");
42                  Alumno refInsertar = lista.buscarPorCodigo(codRefIns);
43                  if (refInsertar == null) {
44                      System.out.println("El codigo de referencia no existe.");
45                      break;
46                  }
47                  lista.insertarPorReferencia(refInsertar, leerAlumno());
48                  System.out.println("Alumno insertado despues de " + codRefIns);
49                  break;
```

```
            case 5:
                mostrarEliminado(lista.eliminarInicio());
                break;
            case 6:
                mostrarEliminado(lista.eliminarFinal());
                break;
            case 7:
                mostrarEliminado(lista.eliminarEnPosicion(pedirEntero("Posicion (desde 1): ")));
                break;
            case 8:
                String codRefEliminar = pedirTexto("Codigo del alumno a eliminar: ");
                Alumno refEliminar = lista.buscarPorCodigo(codRefEliminar);
                mostrarEliminado(lista.eliminarPorReferencia(refEliminar));
                break;
            case 9:
                Alumno encontrado = lista.buscarPorCodigo(pedirTexto("Codigo a buscar: "));
                System.out.println(encontrado == null ? "Alumno no encontrado." : encontrado);
                break;
            case 10:
                System.out.println("\n" + lista.mostrarTodos());
                break;
            case 11:
                lista.ordenar(false);
                System.out.println("Ordenado por codigo (A-Z).\n" + lista.mostrarTodos());
                break;
            case 12:
                lista.ordenar(true);
                System.out.println("Ordenado por promedio (mayor a menor).\n" + lista.mostrarTodos());
                break;
            case 13:
                String codigo1 = pedirTexto("Codigo del primer alumno: ");
                String codigo2 = pedirTexto("Codigo del segundo alumno: ");
                System.out.println(lista.intercambiar(codigo1, codigo2)
                        ? "Alumnos intercambiados." : "Uno o ambos codigos no existen.");
                break;
            case 14:
                System.out.println("Cantidad de alumnos: " + lista.obtenerTamanio());
                break;
            case 15:
                String codigo = pedirTexto("Codigo del alumno a modificar: ");
                if (lista.buscarPorCodigo(codigo) == null) {
                    System.out.println("Alumno no encontrado.");
                    break;
                }
                String nombre = pedirTexto("Nuevo nombre: ");
                int edad = pedirEntero("Nueva edad: ");
```

```
96                 double promedio = pedirDecimal("Nuevo promedio (0 a 20): ");
97                 lista.modificar(codigo, nombre, edad, promedio);
98                 System.out.println("Alumno modificado. Se conserva su codigo.");
99                 break;
100            case 16:
101                System.out.println("La lista esta vacia: " + lista.estaVacia());
102                break;
103            case 0:
104                System.out.println("Saliendo del sistema...");
105                break;
106            default:
107                System.out.println("Opcion no valida.");
108        }
109    }
110
111    private static Alumno leerAlumno() {
112        String codigo = pedirTexto("Codigo: ");
113        String nombre = pedirTexto("Nombre: ");
114        int edad = pedirEntero("Edad: ");
115        double promedio = pedirDecimal("Promedio (0 a 20): ");
116        return new Alumno(codigo, nombre, edad, promedio);
117    }
118
119    private static void mostrarEliminado(Alumno alumno) {
120        System.out.println(alumno == null ? "La lista esta vacia o no se encontro el alumno."
121                : "Alumno eliminado: " + alumno);
122    }
123
124    private static String pedirTexto(String mensaje) {
125        System.out.print(mensaje);
126        return teclado.nextLine().trim();
127    }
128
129    private static int leerEntero() {
130        String texto = teclado.nextLine().trim();
131        try {
132            return Integer.parseInt(texto);
133        } catch (NumberFormatException ex) {
134            System.out.println("Debe ingresar un numero entero.");
135            return -1;
136        }
137    }
138
139    private static int pedirEntero(String mensaje) {
140        String texto = pedirTexto(mensaje);
```

```
141         try {
142             return Integer.parseInt(texto);
143         } catch (NumberFormatException ex) {
144             throw new IllegalArgumentException("Debe ingresar un numero entero.");
145         }
146     }
147 
148     private static double pedirDecimal(String mensaje) {
149         String texto = pedirTexto(mensaje).replace(',', '.');
150         try {
151             return Double.parseDouble(texto);
152         } catch (NumberFormatException ex) {
153             throw new IllegalArgumentException("Debe ingresar un numero decimal valido.");
154         }
155     }
156 }
```

17 Materiales de referencia

- Proyecto proporcionado: CSemana10.zip (paquetes Modelo, DAO y Vista).
- Enunciado 01 del laboratorio de Listas Simplemente Enlazadas, Semana 10.
- Informe de ejemplo: «Informe del enunciado dado de listas simples enlazadas» (estructura tomada como modelo).
- Documentación oficial de Java SE: clases String, StringBuilder, Double (método isFinite) y excepción IllegalArgumentException.
- Material del curso Algoritmos y Estructura de Datos, Escuela Profesional de Ingeniería de Sistemas y Computación, UPLA.